

React, state, forms & data, from zero to exam-ready

Everything the exam can ask you to write by hand: TypeScript basics, array and object methods, nested data, React components, state, props and forms. It's built around the exact tasks in your repo, and every solution here was run against your course's own tests.

Exam: **Wed, Sep 30**

Format: **on paper, write code**

Focus: **React + data methods**

Solutions verified: **81 course tests passing**

Contents

START HERE

How to use this guide	3
Study roadmap (4 days + mock)	4

PART 1 • FOUNDATIONS

1 • TypeScript basics	7
2 • Arrays and higher-order methods	12
3 • Objects, interfaces and spread	19
4 • Nested data	26

PART 2 • REACT

5 • React and JSX basics	34
6 • State with useState	38
7 • Props and lifting state	47
8 • Forms	57

PART 3 • PUTTING IT TOGETHER

9 • Reading specs and tests	67
10 • Writing a component on paper	69

PART 4 • PRACTICE

11 • Practice: data functions	71
12 • Practice: components	73
13 • Spot the bug and trace it	75

PART 5 • REFERENCE

14 • Mistakes checklist	79
15 • Cheat sheet	80
16 • Glossary	84
Answer key	88

Printable edition of the online guide. Practice solutions are collected in the Answer Key at the back so you can work each problem on paper first.

START HERE

How to use this guide

Your exam is on paper, so reading code is not enough. You have to be able to **produce** code from a blank page. Every part of this guide is set up for that.

1. **Read a part once, slowly.** Type or trace the small examples in your head.
2. **Close the guide and write the task solutions by hand.** Use real paper. Include the imports, the types and the `return`.
3. **Compare with the verified solution.** Circle every difference. Most points are lost on small things like a missing `key`, a missing spread, or `onClick={f()}` instead of `onClick={f}`.
4. **Redo the ones you missed the next day.** Spaced repetition beats re-reading.

WHAT YOUR TASKS SAY THE EXAM WILL LOOK LIKE

Your repo has 9 task branches: functions → arrays → objects → nested → state → components → forms. The exam focuses on the last part of that chain, so expect questions like these:

- **Write a data function** over a `Question[]` using `map`, `filter`, `reduce`, `find`, `every`, spread, and no mutation.
- **Write a component with state:** buttons that change `useState` values, conditional rendering, disabled buttons.
- **Fix a broken component:** lift state up, pass props, stop mutating, stop calling components like functions.
- **Write a form:** a controlled textbox, number box, checkbox, switch, radio group or select, using React Bootstrap `Form` components.
- **Combine them:** a component that holds an array of objects in state and updates one item by id from a button or form.

THE 5 TEMPLATES TO MEMORIZE

If you can write these five from memory, you can solve almost any question on the exam. Each one is in the [cheat sheet](#).

1. Update one item in an array by id: `arr.map(x => x.id === id ? {...x, field: v} : x)`
2. A component with `useState` and a button
3. A controlled input: `value={state}` plus `onChange={(e) => setState(e.target.value)}`
4. Lifting state: the parent owns `useState` and passes the value and setter down as props
5. Rendering a list: `{items.map(item => <li key={item}>{item}</li>)}`

START HERE

Study roadmap

The core plan is four focused days, then a mock-exam day and a light warm-up on exam morning. Each day builds on the one before. Don't skip ahead to React before arrays feel automatic, because every React question uses them. Tick the boxes with a pen as you go.

Each day ends with a **gate**, a short test that tells you whether you're ready to move on. If you can't pass it, spend the first 30 minutes of the next day finishing it.

Day 0 • Warm-up (optional)

Thu 9/24 • ~1 hr

Get the vocabulary down so tomorrow goes faster.

- ☐ Read [Section 1: TypeScript basics](#) (types, functions, arrow functions, ternaries, string methods) 40 min
- ☐ Skim the [cheat sheet](#) and the [glossary](#) so the words look familiar 20 min

Gate: write `convertYesNo` and `add3` from memory, with types.

Day 1 • Arrays and objects

Fri 9/25 • ~3 hrs

The higher-order methods and immutable updates. Everything in React sits on top of this.

- ☐ Study [Section 2: Arrays](#), especially the method table and "mutating vs non-mutating" 60 min
- ☐ Handwrite all 9 array tasks from memory, then check them 40 min
- ☐ Study [Section 3: Objects](#) (interfaces, spread, destructuring, `Record`) 45 min
- ☐ Handwrite the 10 object tasks, especially `addOption`, `duplicateQuestion` and `mergeQuestion` 35 min

Gate: without looking, say what each of `map`, `filter`, `reduce`, `find`, `findIndex`, `every` and `some` returns, and write `{ ...question, options: [...question.options, x] }` correctly.

Day 2 • Nested data and React basics

Sat 9/26 • ~3.5 hrs

Arrays of objects, then your first components and JSX.

- ☐ Study [Section 4: Nested data](#) and memorize the 8 patterns 60 min
- ☐ Handwrite `renameQuestionById`, `changeQuestionTypeById`, `editOption` and `duplicateQuestionInArray` 45 min
- ☐ Do data practice problems P1–P6 on paper 30 min
- ☐ Study [Section 5: React and JSX basics](#) (components, JSX rules, lists and keys, conditional rendering) 45 min

Gate: write a component that takes `names: string[]` as a prop and renders them in a `<ul>` with keys, and write "update item by id" from memory.

Day 3 • State and components

Sun 9/27 • ~3.5 hrs

`useState`, event handlers, props and lifting state. This is the heart of the exam.

- ☐ Study [Section 6: State](#), then handwrite Counter, RevealAnswer, StartAttempt and TwoDice 75 min
- ☐ Study [Section 7: Components, props and lifting state](#) 45 min
- ☐ Fix all 4 "bad components" on paper (ChooseTeam, ColoredBox, DoubleHalf, ShoveBox) before looking at the answers 45 min
- ☐ Practice problems R1, R4 and the [spot-the-bug](#) set 30 min

Gate: explain in one sentence each why these are wrong: `team.push(x)`, `useState` outside a component, `const box = MoveableBox()`, `onClick={setX(1)}`.

Day 4 • Forms

Mon 9/28 • ~3.5 hrs

Controlled components with React Bootstrap: textbox, number, checkbox, switch, radio, select.

- ☐ Study [Section 8: Forms](#), especially the event-type table and the `value` vs `checked` rule 60 min
- ☐ Handwrite all 5 form tasks (CheckAnswer, GiveAttempts, EditMode, MultipleChoiceQuestion, ChangeColor) 75 min
- ☐ Read [Section 9: Reading the tests](#) so you can turn a spec into the right elements 20 min
- ☐ Practice problems R2, R3, R6 40 min

Gate: from memory, write a radio group built with `.map` and a `Form.Select` built with `.map`, with correct event types.

Day 5 • Mock exam and repair

Tue 9/29 • ~3 hrs

Simulate the real thing, then fix what broke.

- ☐ Timed on paper (75 min): P7–P12, R5, R7, plus any 2 task components picked at random 75 min
- ☐ Grade yourself against the solutions using the [mistakes checklist](#) 30 min
- ☐ Redo every problem you got wrong, from a blank page 45 min
- ☐ Write the whole [cheat sheet](#) from memory once 20 min

Gate: you can write all 5 templates in under 10 minutes total.

Exam day • Warm-up only

Wed 9/30 • 30 min

Don't learn anything new. Warm up your hand.

- ☐ Write the 5 templates once from memory 15 min
- ☐ Read the [mistakes checklist](#) once 10 min

During the exam: read each question twice. Write the function signature and types first. Before moving on, check for keys, spreads, braces and `return`.

TypeScript basics

JavaScript is the language browsers run. **TypeScript** is JavaScript plus types: labels that say what kind of value a variable holds. TypeScript checks those labels before your code runs and gets turned into plain JavaScript. On the exam, you're expected to write the types.

Primitive types

Type	What it holds	Examples
<code>number</code>	Any number: whole or decimal. There's no separate int type.	<code>42</code> , <code>2.5</code> , <code>-1</code> , <code>NaN</code>
<code>string</code>	Text, in <code>"double"</code> , <code>'single'</code> or <code>`backtick`</code> quotes	<code>"Ada"</code> , <code>""</code>
<code>boolean</code>	Exactly <code>true</code> or <code>false</code>	<code>true</code>
<code>null</code>	"Intentionally nothing." You return it on purpose.	<code>null</code>
<code>undefined</code>	"Never set." It's what you get from a missing field or a failed <code>find</code> .	<code>undefined</code>

`NaN` means "Not a Number". You get it from bad math like `parseInt("hello")`. It is a `number`, and it is not equal to anything, *not even itself*. So `x === NaN` is always false. Use `isNaN(x)` or `Number.isNaN(x)` instead.

Variables: `const` vs `let`

```
const quizTitle: string = "TypeScript Fundamentals"; // can never be reassigned
let currentScore: number = 0;                       // can be reassigned
currentScore = 10;                                   // fine
// quizTitle = "New";                               // ERROR: const can't be reassigned

const inferred = 42;                                // TypeScript infers the type number, so no annotation is needed
```

Use `const` by default and `let` only when you need to reassign. Don't use `var`. `const` stops you from *reassigning the variable*, but you can still change what's inside a `const` array or object (for example `arr.push()`). That's why we avoid mutation on purpose.

Union types and type aliases

A **union** (`|`) says the value can be one of several types. A **literal type** limits it to exact values. A **type alias** gives the type a name.

```

let value: string | number;      // either one
value = "Hello";
value = 42;

type LetterGrade = "A" | "B" | "C" | "D" | "F"; // only these 5 strings allowed
let grade: LetterGrade = "A";

// From your repo (src/interfaces/question.ts):
export type QuestionType = "multiple_choice_question" | "short_answer_question";

// Functions can return a union too:
function convertYesNo(word: string): boolean | null { /* ... */ }

```

Functions

A function takes typed **parameters** and has a typed **return type** after the `)`. `void` means it returns nothing.

```

// 1. Function declaration
function add(first: number, second: number): number {
  return first + second;
}

// 2. Arrow function with a body. Needs the word return.
const subtract = (first: number, second: number): number => {
  return first - second;
};

// 3. Arrow function with an expression body. The value is returned automatically.
const multiply = (first: number, second: number): number => first * second;

// 4. Returns nothing
const sayHello = (name: string): void => {
  console.log("Hello", name);
};

// 5. export makes it importable from other files
export function double(n: number): number {
  return n * 2;
}

// 6. The type of a function (used for props later!)
let myOperation: (first: number, second: number) => number;
let setVisible: (newVisibility: boolean) => void;

```

THE ARROW-FUNCTION OBJECT TRAP

To return an object from a short arrow function, **wrap it in parentheses**. Otherwise the `{` is read as the start of a function body.

```

const bad = (n: number) => { value: n }; // WRONG: returns undefined
const good = (n: number) => ({ value: n }); // RIGHT: returns { value: n }

```

Operators

Operator	Meaning	Example → result
<code>+ - * /</code>	Math. <code>+</code> also joins strings.	<code>"a" + 1 → "a1"</code>
<code>%</code>	Remainder (modulo). Great for wrapping around.	<code>(2 + 1) % 3 → 0</code>
<code>=== / !==</code>	Strictly equal / not equal. Always use these , never <code>==</code> .	<code>1 === 1 → true</code>
<code>< > <= >=</code>	Compare	<code>3 >= 3 → true</code>
<code>&&</code>	AND: true only if both sides are true	<code>true && false → false</code>
<code> </code>	OR: true if either side is true	<code>true false → true</code>
<code>!</code>	NOT: flips a boolean. Used to toggle state.	<code>!true → false</code>
<code>cond ? a : b</code>	Ternary: an if/else that <i>produces a value</i>	<code>5 > 3 ? "yes" : "no" → "yes"</code>
<code>+=, -=</code>	Add or subtract into a variable	<code>sum += 5</code>

Truthy and falsy

In conditions, some values count as false. They're called **falsy**: `false`, `0`, `""` (empty string), `null`, `undefined` and `NaN`. **Everything else is truthy**, including every object and every array, even `[]`.

This explains two tricks you'll see:

- `questions.find(...) || null`: if `find` gives `undefined` (falsy), you get `null`.
- `parseInt(released) - 1 || 0`: if the math gives `NaN` (falsy), you get `0`.

If / else and the ternary

```
const age: number = 29;
if (age >= 21) {
  console.log("Can drink");
} else if (age >= 18) {
  console.log("Getting closer");
} else {
  console.log("Cannot drink");
}

// The ternary is an EXPRESSION: it produces a value you can return or store
const label = age >= 21 ? "Can drink" : "Cannot drink";

// Nested ternary (legal, but keep it readable)
const light = color === "red" ? "green" : color === "green" ? "yellow" : "red";
```

You'll use ternaries all the time in React, because JSX needs *expressions*, not statements. You can't put an `if` inside `{ }` in JSX, but you can put a ternary there.

Template literals

```
const name = "Dr. Bart";
const pets = 3;
const message = `${name} has ${pets} pets. He wants ${pets + 1}!`;
// "Dr. Bart has 3 pets. He wants 4!"
```

Backticks let you drop any expression into a string with `${ }`. `"\n"` inside a string is a newline. The `toMarkdown` and `toCSV` tasks use both.

String methods you need

Strings are **immutable**, so every method below returns a *new* string and leaves the original alone.

Method	Does	Example → result
<code>s.length</code>	Number of characters (a property, so no <code>()</code>)	<code>"cat".length</code> → <code>3</code>
<code>s.toUpperCase()</code>	ALL CAPS	<code>"hi".toUpperCase()</code> → <code>"HI"</code>
<code>s.toLowerCase()</code>	all lowercase	<code>"YeS".toLowerCase()</code> → <code>"yes"</code>
<code>s.trim()</code>	Removes spaces from both ends	<code>" 4 ".trim()</code> → <code>"4"</code>
<code>s.endsWith(x)</code>	Does it end with x?	<code>"why?".endsWith("?")</code> → <code>true</code>
<code>s.startsWith(x)</code>	Does it start with x?	<code>"\$5".startsWith("\$")</code> → <code>true</code>
<code>s.includes(x)</code>	Does it contain x anywhere?	<code>"hello".includes("ell")</code> → <code>true</code>
<code>s.slice(start, end)</code>	Characters from start up to (not including) end	<code>"My First Question".slice(0, 10)</code> → <code>"My First Q"</code>
<code>s.replace(a, b)</code>	Replaces the <i>first</i> a with b	<code>"\$12".replace("\$", "")</code> → <code>"12"</code>
<code>s.split(sep)</code>	String → array	<code>"a,b".split(",")</code> → <code>["a","b"]</code>
<code>s[i]</code>	Character at index i	<code>"cat"[0]</code> → <code>"c"</code>

Numbers from strings, and Math

```
parseInt("42");           // 42
parseInt("12abc");         // 12   (reads digits until it hits a non-digit)
parseInt("hello");         // NaN
parseInt("");              // NaN   (important for form number boxes!)
isNaN(parseInt("x"));      // true, so this is how you check for a failed parse

Math.floor(4.7);           // 4    (rounds down)
Math.round(4.5);          // 5
Math.random();             // a random number from 0 up to (not including) 1
1 + Math.floor(Math.random() * 6); // a random die roll, 1 to 6 (TwoDice's d6)
```

Your `functions.ts` tasks, solved

fahrenheitToCelsius, add3, shout, isQuestion, convertYesNo

`src/functions.ts`

Convert F to C; add only the positive numbers; uppercase plus "!"; check for a trailing "?" without an if; map "yes"/"no" (any case) to true/false, anything else to null.

`src/functions.ts` · model solution (passes course tests)

```
export function fahrenheitToCelsius(temperature: number): number {
  return ((temperature - 32) * 5) / 9;
}

export function add3(first: number, second: number, third: number): number {
  let sum = 0;
  if (first > 0) {
    sum += first;
  }
  if (second > 0) {
    sum += second;
  }
  if (third > 0) {
    sum += third;
  }
  return sum;
}

export function shout(message: string): string {
  return message.toUpperCase() + "!";
}

export function isQuestion(message: string): boolean {
  return message.endsWith("?");
}

export function convertYesNo(word: string): boolean | null {
  const lower = word.toLowerCase();
  if (lower === "yes") {
    return true;
  } else if (lower === "no") {
    return false;
  } else {
    return null;
  }
}
```

- `isQuestion` returns `message.endsWith("?")` directly. That's already a boolean, so there's no need for `? true : false`.
- `convertYesNo` lowercases once and then compares. The return type `boolean | null` is a union.

Arrays and higher-order methods

An **array** is an ordered list. Its type is the element type followed by `[]`: `number[]`, `string[]`, `Question[]`.

```
const grades: number[] = [90, 93, 78];
grades[0];                // 90    (indexes start at 0)
grades[grades.length - 1]; // 78    (last element)
grades.length;            // 3      (property, no parentheses)
grades[10];               // undefined (out of range doesn't crash)
```

Reference vs value: why "just copy it" matters

A variable holding an array doesn't hold the items. It holds a *reference*, which is like an address pointing to the array. Two variables can point to the same array.

```
const x = [1, 2, 3];
const z = x;           // z points to the SAME array
x.push(4);
console.log(z);        // [1, 2, 3, 4]: z "changed" too!
console.log(x === z);  // true: same reference

const y = [1, 2, 3, 4];
console.log(x === y);  // false: same contents, different arrays

const copy = [...x];   // spread makes a NEW array with the same items
console.log(copy === x); // false
```

WHY THIS MATTERS FOR REACT

React decides whether to re-render by checking *whether the reference changed*, not whether the contents changed. If you `push` into a state array and hand back the same array, React sees the same reference and **does nothing**. So in this course you never mutate. You always build a *new* array.

The spread operator `...`

Spread "unpacks" an array's items into a new array literal.

```
const prices = [10, 8, 9];
const atEnd   = [...prices, 55];           // [10, 8, 9, 55]
const atStart = [55, ...prices];           // [55, 10, 8, 9]
const doubled = [...prices, ...prices];    // [10, 8, 9, 10, 8, 9]
const copy    = [...prices];               // [10, 8, 9] (new array)
```

How a higher-order method works

A **higher-order function** is a function that takes another function as input. `map`, `filter`, `reduce` and the rest take a **callback**, a small function that the method calls *once per element*.

```
const prices = [13, 7, 8, 2];

//           └─ callback: runs once per element ─┘
const doubled = prices.map((price: number): number => price * 2);
//           ^ each element ^ return type ^ what to produce

// The callback can also get the index as a second parameter:
prices.map((price: number, index: number): string => index + ": " + price);
// ["0: 13", "1: 7", "2: 8", "3: 2"]

// You can pass a named function instead. Pass the NAME, don't call it:
const isLow = (price: number): boolean => price < 10;
prices.filter(isLow); // RIGHT
// prices.filter(isLow()); // WRONG: that calls isLow immediately with no argument
```

The method table: learn this cold

Method	Callback returns	Method returns	Use it to...
<code>map</code>	the new value	new array, same length	transform every item
<code>filter</code>	boolean (keep?)	new array, same or shorter	keep only matching items / remove items
<code>reduce</code>	the new running total	one value (any type)	sum, count, find max, build an object
<code>find</code>	boolean	first match or <code>undefined</code>	get one item (by id)
<code>findIndex</code>	boolean	index of first match or <code>-1</code>	know <i>where</i> an item is
<code>every</code>	boolean	true if all pass (empty → true)	"are all...?"
<code>some</code>	boolean	true if any pass (empty → false)	"is there at least one...?"
<code>includes(x)</code>	(no callback)	boolean	"is x in the array?"
<code>indexOf(x)</code>	(no callback)	index or <code>-1</code>	position of a plain value
<code>join(sep)</code>	(no callback)	string	<code>[1,2,3].join("+")</code> → <code>"1+2+3"</code>
<code>slice(s, e)</code>	(no callback)	new array (copy of part)	take the first n, everything before an index
<code>flatMap</code>	an array	new array, flattened one level	turn one item into 0, 1 or 2 items
<code>concat(arr)</code>	(no callback)	new array	same as <code>[...a, ...b]</code>

Each method with an example

```
const prices = [13, 7, 8, 2, 19];

prices.map((p: number): number => p * 2);           // [26, 14, 16, 4, 38]
prices.map((p: number): string => "$" + p);          // ["$13", "$7", "$8", "$2", "$19"]
prices.filter((p: number): boolean => p < 10);       // [7, 8, 2]
prices.find((p: number): boolean => p < 10);         // 7 (first match only)
prices.find((p: number): boolean => p > 100);        // undefined
prices.findIndex((p: number): boolean => p < 10);    // 1
prices.findIndex((p: number): boolean => p > 100);   // -1
prices.every((p: number): boolean => p < 10);       // false
prices.some((p: number): boolean => p < 10);        // true
prices.includes(8);                                 // true
prices.join(" and ");                               // "13 and 7 and 8 and 2 and 19"
prices.slice(0, 2);                                 // [13, 7]
prices.slice(1);                                    // [7, 8, 2, 19]

// Conditional change with map (same length, some items changed)
prices.map((p: number): number => (p < 10 ? 2 * p : p)); // [13, 14, 16, 4, 19]
```

reduce , step by step

`reduce` walks the array carrying an **accumulator** (a running result). The callback takes (`accumulator`, `currentItem`) and returns the *next* accumulator. The second argument to `reduce` is the **starting value**. Always give one.

```
const nums = [13, 7, 8, 2];
const sum = nums.reduce(
  (currentTotal: number, num: number): number => currentTotal + num,
  0, // starting value
);
// Step by step: 0 + 13 = 13 → 13 + 7 = 20 → 20 + 8 = 28 → 28 + 2 = 30
// sum === 30

// Count how many pass a test (or just use filter(...).length)
const lowCount = nums.reduce((count: number, n: number): number => (n < 10 ? count + 1 : count),
  0);

// Find the biggest
const biggest = nums.reduce((best: number, n: number): number => (n > best ? n : best), nums[0]);
```

REDUCE GOTCHA

Without a starting value, `{}.reduce(...)` **crashes**. Always pass the starting value: `0` for sums, `""` for strings, `{}` for objects, `[]` for arrays.

Chaining

Each method returns a new array, so you can call the next method right away. Read it top to bottom like a pipeline:

```
const messages = ["Hi!", "why?", "ok"];
const result = messages
  .filter((m: string): boolean => !m.endsWith("?")) // ["Hi!", "ok"]
  .map((m: string): string => (m.endsWith("!") ? m.toUpperCase() : m)); // ["HI!", "ok"]
```

Mutating vs non-mutating methods

MUTATE THE ORIGINAL (AVOID ON STATE)

push, pop, shift, unshift, splice, sort, reverse, arr[i] = x

If you need one of these, run it on a **copy**: `const copy = [...arr]; copy.splice(...)`.

RETURN SOMETHING NEW (SAFE)

map, filter, reduce, slice, concat, flatMap, join, spread [...arr]

splice and sort (on a copy)

```
// splice(startIndex, howManyToDelete, ...itemsToInsert). It CHANGES the array.
const copy = [10, 8, 9, 7];
copy.splice(2, 0, 499); // insert 499 at index 2, delete nothing → [10, 8, 499, 9, 7]
copy.splice(1, 1); // delete 1 item at index 1 → [10, 499, 9, 7]
copy.splice(0, 1, 42); // replace the item at index 0 with 42 → [42, 499, 9, 7]

// sort mutates too, so copy first. For numbers, pass a compare function:
const ascending = [...copy].sort((a: number, b: number): number => a - b);
const descending = [...copy].sort((a: number, b: number): number => b - a);
// Without a compare function, sort compares as STRINGS: [10, 9, 100].sort() → [10, 100, 9]
```

Your arrays.ts tasks, solved and explained

These are model solutions. Each one passes the tests in your repo. Try each one on paper first.

bookEndList src/arrays.ts

Return just the first and last number. Empty → []. One element → that number twice.

```
export function bookEndList(numbers: number[]): number[] {
  if (numbers.length === 0) {
    return [];
  }
  return [numbers[0], numbers[numbers.length - 1]];
}
```

With one element, index `0` and index `length - 1` are the same spot, so you get it twice for free.

tripleNumbers

src/arrays.ts

```
export function tripleNumbers(numbers: number[]): number[] {
  return numbers.map((num: number): number => num * 3);
}
```

stringsToIntegers and removeDollars

src/arrays.ts

Parse each string to an integer; if it can't be parsed, use 0. `removeDollars` also strips a leading "\$".

```
export function stringsToIntegers(numbers: string[]): number[] {
  return numbers.map((str: string): number => {
    const parsed = parseInt(str);
    return isNaN(parsed) ? 0 : parsed;
  });
}

export const removeDollars = (amounts: string[]): number[] => {
  return amounts.map((amount: string): number => {
    const noDollar = amount.startsWith("$") ? amount.slice(1) : amount;
    const parsed = parseInt(noDollar);
    return isNaN(parsed) ? 0 : parsed;
  });
};
```

The pattern is: map each string, parse it, then `isNaN(parsed) ? 0 : parsed`. `removeDollars` is written as an arrow function stored in a `const`. That's the same thing as a function declaration.

shoutIfExclaiming

src/arrays.ts

Remove strings ending in "?", uppercase strings ending in "!".

```
export const shoutIfExclaiming = (messages: string[]): string[] => {
  return messages
    .filter((message: string): boolean => !message.endsWith("?"))
    .map((message: string): string =>
      message.endsWith("!") ? message.toUpperCase() : message,
    );
};
```

"Remove" means `filter`. "Change some" means `map` with a ternary.

countShortWords and allRGB

src/arrays.ts

```
export function countShortWords(words: string[]): number {
  return words.filter((word: string): boolean => word.length < 4).length;
}

export function allRGB(colors: string[]): boolean {
  return colors.every(
    (color: string): boolean =>
      color === "red" || color === "blue" || color === "green",
  );
}
```

To count matches, use `filter(...).length`. For "are ALL..." use `every`. It already returns `true` for an empty list, just like the spec asks.

makeMath

src/arrays.ts

`[1, 2, 3] → "6=1+2+3". [] → "0=0".`

```
export function makeMath(addends: number[]): string {
  if (addends.length === 0) {
    return "0=0";
  }
  const sum = addends.reduce(
    (total: number, num: number): number => total + num,
    0,
  );
  return sum + "=" + addends.join("+");
}
```

injectPositive

src/arrays.ts

After the FIRST negative number, insert the sum of all numbers before it. If there's no negative, append the total sum.

```

export function injectPositive(values: number[]): number[] {
  const negativeIndex = values.findIndex((num: number): boolean => num < 0);
  if (negativeIndex === -1) {
    const total = values.reduce(
      (sum: number, num: number): number => sum + num,
      0,
    );
    return [...values, total];
  }
  const sumBefore = values
    .slice(0, negativeIndex)
    .reduce((sum: number, num: number): number => sum + num, 0);
  const result = [...values];
  result.splice(negativeIndex + 1, 0, sumBefore);
  return result;
}

```

This is the hardest array task. It combines `findIndex` (where), `slice` (everything before), `reduce` (sum) and `splice` on a `copy` (insert).

Objects, interfaces and spread

An **object** groups named values, called **fields** or **properties**, inside `{ }`. You read a field with a dot.

```
const cisc275 = { id: "CISC275", seats: 80, online: false, labs: ["20", "21"] };
cisc275.seats;      // 80
cisc275.labs[0];    // "20"
cisc275.Seats;      // undefined, and TypeScript flags it. Field names are case-sensitive.
```

Interfaces: describing the shape of an object

An **interface** is a type for objects. It lists every field and its type. An object labeled with that interface must have exactly those fields.

```
interface Dog {
  name: string;
  age: number;
  breed: "Corgi" | "Chihuahua" | "Mutt"; // a field can be a union of literals
  fuzzy: boolean;
  nickname?: string; // ? = optional field (may be missing)
}

const ada: Dog = { name: "Ada Bart", breed: "Corgi", age: 4, fuzzy: true };
```

These are the interfaces from your repo. **Know them by heart**, because exam questions are built on them.

src/interfaces/question.ts

```
/** QuestionType influences how a question is asked and what kinds of answers are possible */
export type QuestionType = "multiple_choice_question" | "short_answer_question";

/** A representation of a Question in a quizzing application */
export interface Question {
  /** A unique identifier for the question */
  id: number;
  /** The human-friendly title of the question */
  name: string;
  /** The instructions and content of the Question */
  body: string;
  /** The kind of Question; influences how the user answers and what options are displayed */
  type: QuestionType;
  /** The possible answers for a Question (for Multiple Choice questions) */
  options: string[];
  /** The actually correct answer expected */
  expected: string;
  /** How many points this question is worth, roughly indicating its importance and difficulty */
  points: number;
  /** Whether or not this question is ready to display to students */
  published: boolean;
}
```

```
src/interfaces/answer.ts
```

```
/**
 * A representation of a students' answer in a quizzing game
 */
export interface Answer {
  /** The ID of the question being answered. */
  questionId: number;
  /** The text that the student entered for their answer. */
  text: string;
  /** Whether or not the student has submitted this answer. */
  submitted: boolean;
  /** Whether or not the students' answer matched the expected. */
  correct: boolean;
}
```

`type` vs `interface`: both can describe an object. In this course, `interface` is used for object shapes (`Question`) and `type` for unions and aliases (`QuestionType`).

Immutable updates with object spread

`{ ...obj }` copies every field of `obj` into a **new** object. Any fields listed *after* the spread override the copied ones.

```
const myPhone = { brand: "Samsung", model: "Galaxy", batteryLeft: 97 };

// BAD (mutation): changes the original
myPhone.batteryLeft = 48;

// GOOD (new object): original untouched
const usedPhone = { ...myPhone, batteryLeft: 48 };
// { brand: "Samsung", model: "Galaxy", batteryLeft: 48 }

// Several changes at once, based on the old values
const player = { score: 100, money: 120, position: 9 };
const advanced = { ...player, score: player.score * 2, position: player.position + 1 };
// { score: 200, money: 120, position: 10 }

// Toggle a boolean field
const flipped = { ...question, published: !question.published };
```

ORDER MATTERS

`{ ...q, name: "New" }` gives you the new name. `{ name: "New", ...q }` gives you the **old** name, because the spread comes last and overwrites it. Always put the spread first and the changes after.

Nested fields: spread is shallow

Spread copies only the top level. If a field holds an array or object, the copy **shares** that same inner array. To change a nested array, make a new one too.

```
const student = { name: "Ada", grades: [100, 99, 78] };

// WRONG: shares the grades array, so push changes BOTH students
const bad = { ...student };
bad.grades.push(100);

// RIGHT: new outer object AND new inner array
const withNewGrade = { ...student, grades: [...student.grades, 100] };
```

This is exactly the `addOption` task: `{ ...question, options: [...question.options, newOption] }`.

Shorthand properties and computed keys

```
const id = 5;
const name = "Q5";
const q1 = { id: id, name: name }; // long form
const q2 = { id, name };           // shorthand: same thing when the variable name = field name

const key = "vanilla";
const votes = { ...oldVotes, [key]: oldVotes[key] + 1 }; // [key] uses the VALUE of key as the field name
```

Destructuring

Destructuring pulls fields out of an object, or items out of an array, into variables. React uses it constantly: `const [value, setValue] = useState(...)` is array destructuring, and `function Box({ color }: ...)` is object destructuring.

```
const ada = { name: "Ada Bart", breed: "Corgi", age: 4 };

const { name, breed } = ada; // name = "Ada Bart", breed = "Corgi"
const { name: dogName } = ada; // rename while pulling out: dogName = "Ada Bart"

const rgb = [255, 128, 200];
const [red, green, blue] = rgb; // red = 255, green = 128, blue = 200
const [, second] = rgb; // skip the first: second = 128

// In function parameters: take only the field you need
function checkIfCorgi({ breed }: { breed: string }): boolean {
  return breed === "Corgi";
}
```

Record : a lookup table

`Record<KeyType, ValueType>` is an object used as a dictionary or map. Looking something up by key is instant, with no searching.

```

const courseLookup: Record<string, string> = {
  CISC108: "Introduction to Computer Science",
  CISC275: "Introduction to Software Engineering",
};
courseLookup["CISC275"]; // "Introduction to Software Engineering"

// From your CycleHoliday task: "what comes next" as a lookup
type Holiday = "🌲" | "🐰" | "🎃" | "👹" | "💖";
const NEXT_BY_YEAR: Record<Holiday, Holiday> = {
  "💖": "🐰", "🐰": "🎃", "🎃": "👹", "👹": "🌲", "🌲": "💖",
};
NEXT_BY_YEAR["🎃"]; // "👹"

Object.keys(courseLookup); // ["CISC108", "CISC275"]
Object.values(courseLookup); // ["Introduction to...", "Introduction to..."]
Object.entries(courseLookup); // [["CISC108", "Intro..."], ["CISC275", "Intro..."]]

// Array → Record with Object.fromEntries (pairs of [key, value])
const byId: Record<number, Question> = Object.fromEntries(
  questions.map((q: Question): [number, Question] => [q.id, q]),
);

```

JSON

JSON is a text format for data. `JSON.stringify(obj)` turns an object into a string, and `JSON.parse(str)` turns the string back into an object. Your tests load `src/data/questions.json` this way. Also remember that two objects with identical contents are **not** `===`, because they're different references.

Your `objects.ts` tasks, solved and explained

makeBlankQuestion `src/objects.ts`

Build a new `Question` with the given `id`, `name` and `type`. `body` and `expected` are `""`, `options` is `[]`, `points` is `1`, `published` is `false`.

```

export function makeBlankQuestion(
  id: number,
  name: string,
  type: QuestionType,
): Question {
  return {
    id: id,
    name: name,
    type: type,
    body: "",
    expected: "",
    options: [],
    points: 1,
    published: false,
  };
}

```

When you're building an object that matches an interface, list **every** field. Leaving one out is a type error.

isCorrect and isValid

src/objects.ts

isCorrect ignores case and surrounding whitespace. isValid: short answer → always valid; multiple choice → answer must be one of the options.

```
export function isCorrect(question: Question, answer: string): boolean {
  return (
    answer.trim().toLowerCase() === question.expected.trim().toLowerCase()
  );
}

export function isValid(question: Question, answer: string): boolean {
  if (question.type === "short_answer_question") {
    return true;
  }
  return question.options.includes(answer);
}
```

toShortForm and toMarkdown

src/objects.ts

"9: My First Q". Markdown: "# name", then the body, then "- option" lines for multiple choice.

```
export function toShortForm(question: Question): string {
  return question.id + ": " + question.name.slice(0, 10);
}

export function toMarkdown(question: Question): string {
  let result = "# " + question.name + "\n" + question.body;
  if (question.type === "multiple_choice_question") {
    const optionLines = question.options.map(
      (option: string): string => "- " + option,
    );
    result = result + "\n" + optionLines.join("\n");
  }
  return result;
}
```

`slice(0, 10)` gives the first 10 characters. For the markdown, `map` each option to a line and `join("\n")` the lines together.

renameQuestion and publishQuestion

src/objects.ts

```
export function renameQuestion(question: Question, newName: string): Question {
  return { ...question, name: newName };
}

export function publishQuestion(question: Question): Question {
  return { ...question, published: !question.published };
}
```

"Return a new version, except..." always means `{ ...old, changedField: newValue }`.

duplicateQuestion

src/objects.ts

Copy everything, but with the new id, the name "Copy of ORIGINAL", and published = false.

```
export function duplicateQuestion(id: number, oldQuestion: Question): Question {
  return {
    ...oldQuestion,
    id: id,
    name: "Copy of " + oldQuestion.name,
    published: false,
    options: [...oldQuestion.options],
  };
}
```

Copying `options` with `[...oldQuestion.options]` gives the duplicate its own array. The tests pass without it, but it's the safe habit and graders like it.

addOption

src/objects.ts

Return a new question with newOption added. The new Question MUST have its own copy of the options list.

```
export function addOption(question: Question, newOption: string): Question {
  return { ...question, options: [...question.options, newOption] };
}
```

mergeQuestion

src/objects.ts

Use body, type, options and expected from contentQuestion; points from the second object; new id and name; published = false.

```

export function mergeQuestion(
  id: number,
  name: string,
  contentQuestion: Question,
  { points }: { points: number },
): Question {
  return {
    ...contentQuestion,
    id: id,
    name: name,
    points: points,
    published: false,
    options: [...contentQuestion.options],
  };
}

```

The fourth parameter is **destructured**: `{ points }: { points: number }` means "take an object that has a `points` field, and give me that field as a variable." Any `Question` fits, because it has a `points` field.

Nested data: arrays of objects

Nested data means structures inside structures: an array of objects (`Question[]`), an object holding an array (`question.options`), or both at once. This is the most important data skill on the exam, because React state is usually an array of objects.

The 8 patterns

Almost every nested-data question is one of these eight, or two of them combined. Here they are with the `Question` interface:

the 8 nested-data patterns

```
// 1. KEEP some items (filter by a field)
const published = questions.filter((q: Question): boolean => q.published);

// 2. PULL OUT one field from each (map to a field)
const names = questions.map((q: Question): string => q.name);

// 3. TOTAL a field (reduce)
const total = questions.reduce((sum: number, q: Question): number => sum + q.points, 0);

// 4. FIND one by id (find), with null if missing
const found = questions.find((q: Question): boolean => q.id === id);
const result = found === undefined ? null : found; // or: found || null

// 5. REMOVE one by id (filter out)
const without = questions.filter((q: Question): boolean => q.id !== id);

// 6. ADD one to the end (spread)
const added = [...questions, newQuestion];

// 7. CHANGE EVERY item (map + object spread)
const allPublished = questions.map((q: Question): Question => ({ ...q, published: true }));

// 8. CHANGE ONE item by id (map + ternary + object spread). THE BIG ONE.
const renamed = questions.map(
  (q: Question): Question => (q.id === targetId ? { ...q, name: newName } : q),
);
```

PATTERN 8 IN PLAIN WORDS

"Go through every question. If it's the one I want, give back a copy with the change. Otherwise give back the question unchanged." The result is a new array of the same length, where only the target is a new object.

Going one level deeper: a nested array inside one item

To change the `options` of one question, you nest pattern 8 with an array operation on its options:

```

// Add an option to question #targetId
questions.map((q: Question): Question =>
  q.id === targetId ? { ...q, options: [...q.options, newOption] } : q,
);

// Remove option number optionIndex from question #targetId
questions.map((q: Question): Question =>
  q.id === targetId
    ? { ...q, options: q.options.filter((opt: string, i: number): boolean => i !==
optionIndex) }
    : q,
);

// Replace option number optionIndex in question #targetId
questions.map((q: Question): Question =>
  q.id === targetId
    ? { ...q, options: q.options.map((opt: string, i: number): string => (i === optionIndex ?
newOption : opt)) }
    : q,
);

```

Inserting at a position

```

// Insert newQuestion right AFTER the question with targetId
const index = questions.findIndex((q: Question): boolean => q.id === targetId);
const copy = [...questions]; // copy first! splice mutates
copy.splice(index + 1, 0, newQuestion); // at index+1, delete 0, insert newQuestion
return copy;

// Same idea with flatMap: each item becomes an ARRAY of 1 or 2 items
questions.flatMap((q: Question): Question[] =>
  q.id === targetId ? [q, newQuestion] : [q],
);

```

Mapping to a different shape

`map` can turn each item into a *different* type. `makeAnswers` turns each `Question` into an `Answer`:

```

questions.map((q: Question): Answer => ({
  questionId: q.id,
  text: "",
  submitted: false,
  correct: false,
})); // remember: ({ ... }) with parentheses to return an object

```

Shallow vs deep copy

```
const tonight = kiki; // NOT a copy, same reference
const shallow = { ...kiki }; // new object, but nested arrays are still shared
const deepList = movies.map((m: Movie): Movie => ({ ...m })); // new array of new objects

// Deep copy every level you might change:
emails.map((email: Email): Email => ({
  ...email,
  tags: [...email.tags],
  sender: { ...email.sender },
  recipients: email.recipients.map((c: Contact): Contact => ({ ...c })),
})));
```

The rule: **anything you change must be new, and so must everything that contains it.** Things you don't change can be reused.

Optional chaining ?.

`find` can return `undefined`, and reading a field of `undefined` crashes. `found?.name` gives `undefined` instead of crashing. You can also check first: `found === undefined ? "none" : found.name`.

Your `nested.ts` tasks, solved and explained

getPublishedQuestions and getNonEmptyQuestions

src/nested.ts

Keep published ones. Keep "non-empty" ones (empty = body "" AND expected "" AND options []).

```
export function getPublishedQuestions(questions: Question[]): Question[] {
  return questions.filter((question: Question): boolean => question.published);
}

export function getNonEmptyQuestions(questions: Question[]): Question[] {
  return questions.filter(
    (question: Question): boolean =>
      question.body !== "" ||
      question.expected !== "" ||
      question.options.length !== 0,
  );
}
```

"Not empty" is the opposite of "body empty AND expected empty AND no options". Flipping an AND gives an OR: at least one of them is filled in.

findQuestion and removeQuestion

src/nested.ts

```
export function findQuestion(
  questions: Question[],
  id: number,
): Question | null {
  const found = questions.find(
    (question: Question): boolean => question.id === id,
  );
  return found === undefined ? null : found;
}

export function removeQuestion(questions: Question[], id: number): Question[] {
  return questions.filter((question: Question): boolean => question.id !== id);
}
```

getNames, sumPoints, sumPublishedPoints

src/nested.ts

```
export function getNames(questions: Question[]): string[] {
  return questions.map((question: Question): string => question.name);
}

export function sumPoints(questions: Question[]): number {
  return questions.reduce(
    (total: number, question: Question): number => total + question.points,
    0,
  );
}

export function sumPublishedPoints(questions: Question[]): number {
  return sumPoints(getPublishedQuestions(questions));
}
```

`sumPublishedPoints` reuses two functions you already wrote. Composing small functions is encouraged.

toCSV

src/nested.ts

Header "id,name,options,points,published", then one line per question, with options as a COUNT.

```
export function toCSV(questions: Question[]): string {
  const rows = questions.map(
    (question: Question): string =>
    `${question.id},${question.name},${question.options.length},${question.points},${question.published}`,
  );
  return ["id,name,options,points,published", ...rows].join("\n");
}
```

makeAnswers and publishAll

src/nested.ts

```
export function makeAnswers(questions: Question[]): Answer[] {
  return questions.map(
    (question: Question): Answer => ({
      questionId: question.id,
      text: "",
      submitted: false,
      correct: false,
    }),
  );
}

export function publishAll(questions: Question[]): Question[] {
  return questions.map(
    (question: Question): Question => ({ ...question, published: true }),
  );
}
```

sameType and addNewQuestion

src/nested.ts

```
export function sameType(questions: Question[]): boolean {
  if (questions.length === 0) {
    return true;
  }
  const firstType = questions[0].type;
  return questions.every(
    (question: Question): boolean => question.type === firstType,
  );
}

export function addNewQuestion(
  questions: Question[],
  id: number,
  name: string,
  type: QuestionType,
): Question[] {
  return [...questions, makeBlankQuestion(id, name, type)];
}
```

`sameType` compares everything to the first question's type. An empty list counts as "all the same".

renameQuestionById `src/nested.ts`

```
export function renameQuestionById(
  questions: Question[],
  targetId: number,
  newName: string,
): Question[] {
  return questions.map(
    (question: Question): Question =>
      question.id === targetId ?
        { ...question, name: newName }
        : question,
  );
}
```

This is pattern 8 exactly. (The `cond ? / :` line breaks are just your repo's Prettier formatting for a ternary.)

changeQuestionTypeById `src/nested.ts`

Change the type; if the new type is NOT multiple choice, options becomes `[]`.

```
export function changeQuestionTypeById(
  questions: Question[],
  targetId: number,
  newQuestionType: QuestionType,
): Question[] {
  return questions.map(
    (question: Question): Question =>
      question.id === targetId ?
        {
          ...question,
          type: newQuestionType,
          options:
            newQuestionType === "multiple_choice_question" ?
              question.options
              : [],
        }
        : question,
  );
}
```

editOption (with a helper) `src/nested.ts`

For question `targetId`: if `targetOptionIndex` is `-1`, add `newOption` to the end; otherwise replace the option at that index.

```

function editOptionList(
  options: string[],
  targetOptionIndex: number,
  newOption: string,
): string[] {
  if (targetOptionIndex === -1) {
    return [...options, newOption];
  }
  const copy = [...options];
  copy.splice(targetOptionIndex, 1, newOption);
  return copy;
}

export function editOption(
  questions: Question[],
  targetId: number,
  targetOptionIndex: number,
  newOption: string,
): Question[] {
  return questions.map(
    (question: Question): Question =>
      question.id === targetId ?
        {
          ...question,
          options: editOptionList(
            question.options,
            targetOptionIndex,
            newOption,
          ),
        }
      : question,
  );
}

```

The helper does the inner-array work, so `editOption` stays as a simple pattern 8. That's what the task comment means by "think about how a helper function can make it simpler."

duplicateQuestionInArray src/nested.ts

Insert a duplicate (using `duplicateQuestion` with `newId`) directly after the target.

```

export function duplicateQuestionInArray(
  questions: Question[],
  targetId: number,
  newId: number,
): Question[] {
  const index = questions.findIndex(
    (question: Question): boolean => question.id === targetId,
  );
  if (index === -1) {
    return [...questions];
  }
  const copy = [...questions];
  copy.splice(index + 1, 0, duplicateQuestion(newId, questions[index]));
  return copy;
}

```

The one-liner version: `questions.flatMap(q => q.id === targetId ? [q, duplicateQuestion(newId, q)] : [q]).`

React and JSX basics

React is a library for building user interfaces out of **components**. A component is just a **function that returns JSX**. JSX (in `.tsx` files) lets you write HTML-like tags directly inside TypeScript.

Your first component

src/components/Greeting.tsx

```
import React from "react";

export function Greeting(): React.JSX.Element {
  const name = "Ada";
  return (
    <div>
      <h3>Hello, {name}!</h3>
      <p>2 + 2 = {2 + 2}</p>
    </div>
  );
}
```

- The name **must start with a capital letter** (`Greeting` , not `greeting`). React treats lowercase tags as HTML.
- The return type is `React.JSX.Element` , which is what your repo uses. The textbook writes `JSX.Element` . Both mean "some JSX".
- `export` lets other files do `import { Greeting } from "../components/Greeting";`
- You **use** a component like a tag: `<Greeting></Greeting>` or `<Greeting />` .

JSX rules (lose points here if you forget)

Rule	Right	Wrong
Return one parent element (or a fragment)	<code><div><h1/><p/></div></code> or <code><>...</></code>	<code>return <h1/><p/></code>
Use <code>{ }</code> to drop in any TypeScript <i>expression</i>	<code>{count + 1}</code> , <code>{isOn ? "on" : "off"}</code>	<code>{if (x) ...}</code> (statements aren't allowed)
<code>className</code> , not <code>class</code>	<code><div className="App"></code>	<code><div class="App"></code>
<code>style</code> takes an object with camelCase keys, hence the double braces	<code>style={{ backgroundColor: "red", marginLeft: "5px" }}</code>	<code>style="background-color: red"</code>
Every tag must close	<code></code> , <code>
</code> , <code><hr /></code>	<code></code>
Comments in JSX	<code>{/* comment */}</code>	<code><!-- comment --></code>

Wrap multi-line JSX in <code>()</code> after <code>return</code>	<code>return (<div>...</div>);</code>	a bare <code>return</code> followed by a new line (returns <code>undefined</code>)
Event props are camelCase and take a function	<code>onClick={handleClick}</code>	<code>onClick="handleClick()"</code>

HTML elements you'll use

```

<div>...</div>           // generic box (block)
<span>...</span>          // generic inline wrapper
<h1>...</h1> ... <h6>...</h6> // headings (role "heading")
<p>...</p>               // paragraph
<ul><li>...</li></ul>     // bulleted list (li has role "listitem")
<ol><li>...</li></ol>     // numbered list
<strong>...</strong>     // bold
 // image (always give alt text)
<hr />                  // horizontal line
<div data-testid="colored-box">...</div> // lets tests find the element

```

React Bootstrap

Your course uses **React Bootstrap**, which gives you ready-styled components. Import what you use:

```

import { Button, Container, Row, Col, Form } from "react-bootstrap";

<Button onClick={handleClick}>Click me</Button>
<Button size="sm" disabled={true}>Small, disabled</Button>

<Container>
  <Row>
    <Col>Left column</Col>
    <Col>Right column</Col>
  </Row>
</Container>

```

`Form` gets its own part ([Section 8](#)).

Rendering lists with `map` and `key`

To show an array, `map` each item to JSX. Each element you produce needs a unique `key` prop so React can tell siblings apart.

```
const names = ["Ada", "Pumpkin", "Babbage"];
return (
  <ul>
    {names.map((name: string) => (
      <li key={name}>{name}</li>
    ))}
  </ul>
);

// Objects: use a unique field (the id!) as the key
{questions.map((q: Question) => (
  <li key={q.id}>{q.name} ({q.points} pts)</li>
))}
```

- The key goes on the **outermost** element returned from `map`.
- Keys must be unique among siblings. Use an id, or the string itself if the strings are unique. The `index ( (item, index) => key={index} )` is a last resort.
- Wrap the `map` in `{ }`, because it's an expression inside JSX.

Conditional rendering: three ways

```
// 1. && : show it, or show nothing
{visible && <div>42</div>}

// 2. Ternary: show one thing or the other
{isCorrect ? "✓" : "✗"}
{editMode ? <Form.Control value={name} onChange={updateName} /> : <span>{name}</span>}

// 3. Decide BEFORE the return, using a normal if or a variable
let message: string;
if (attempts === 0) {
  message = "Out of attempts";
} else {
  message = "Attempts: " + attempts;
}
return <div>{message}</div>;
```

Composing components: App.tsx

Components render other components. That's how your `App.tsx` builds the page:

src/App.tsx (shortened)

```
import React from "react";
import "./App.css";
import { Counter } from "../components/Counter";
import { CheckAnswer } from "../form-components/CheckAnswer";

function App(): React.JSX.Element {
  return (
    <div className="App">
      <header className="App-header">UD CISC275 with React Hooks and TypeScript</header>
      <CheckAnswer expectedAnswer="42"></CheckAnswer>
      <hr />
      <Counter></Counter>
    </div>
  );
}

export default App;
```

`expectedAnswer="42"` passes a **prop**, which is an input to the component. Props are covered fully in [Section 7](#). Strings can be passed with quotes. Anything else goes in braces: `options={["a", "b"]}`, `count={3}`, `visible={true}`.

Model, view, controller



This is React's **one-way data flow**. You never reach into the page to change it. You change the *state*, and React redraws the view from it.

State with `useState`

State is data a component remembers between renders. When state changes, React **re-renders**: it calls your component function again and updates the page to match.

Why a normal variable doesn't work

x broken: always shows 0

```
export function App(): React.JSX.Element {
  let counter: number = 0;           // reset to 0 every time App runs
  function addOne(): void {
    counter += 1;                     // changes the local variable...
  }                                  // ...but React doesn't know, so it never re-renders
  return <Button onClick={addOne}>{counter}</Button>;
}
```

Two problems here: changing a local variable doesn't tell React to redraw, and every render starts over at 0 anyway. `useState` fixes both.

The `useState` syntax

✓ working counter

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

export function App(): React.JSX.Element {
  const [counter, setCounter] = useState<number>(0);
  //   ^value   ^setter           ^type   ^initial value

  function addOne(): void {
    setCounter(counter + 1); // ask React to store a new value and re-render
  }
  return <Button onClick={addOne}>{counter}</Button>;
}
```

- `useState<T>(initial)` returns a pair, which you **destructure** into `[value, setValue]`.
- Name them `thing` / `setThing`.
- Only the **setter** can change state. `counter = 5` is an error (it's `const`), and `counter++` doesn't work either.
- Common types: `useState<number>`, `<string>`, `<boolean>`, `<string[]>`, `<Question[]>`, `<QuestionType>`, `<Record<string, number>>`.

The rules of hooks

TWO RULES. BREAKING THEM IS A CLASSIC EXAM TRAP.

1. Only call `useState` inside a component function, at the top of its body. Never at the top level of a file, and never in a regular helper function. (That's the DoubleHalf bug.)
2. Never call it inside an `if`, a loop, or a nested function. React matches hooks by the order they're called, so the order has to be the same on every render.

Event handlers: pass the function, don't call it

```
<Button onClick={addOne}>+1</Button>           // ✓ pass the function; React calls it on
click
<Button onClick={addOne()}>+1</Button>         // ✗ calls addOne DURING render (infinite
loop)
<Button onClick={() => { setCounter(0); }}>Reset</Button> // ✓ arrow function for inline code or
arguments
<Button onClick={() => { chooseMember(option); }}>Add</Button> // ✓ passing an argument
```

YOUR REPO'S STYLE: BRACES IN HANDLER ARROWS

Your ESLint config has the rule `no-confusing-void-expression`. It rejects `onClick={() => setX(1)}` because a setter returns `void` and that shorthand "returns" it. Write `onClick={() => { setX(1); }}` instead, the way your solved tasks do. Both work in React, but the braces version is the course's style, so use it on paper too.

State updates are not instant

A setter *schedules* a change. The `value` variable doesn't change until the next render.

```
function increaseTwice(): void {
  setValue(1 + value); // value is 0, so this schedules 1
  setValue(1 + value); // value is STILL 0, so this schedules 1 again
}
// result: 1, not 2

function startQuiz(): void {
  setInProgress(true);
  setAttempts(attempts - 1); // fine: each setter changes a different state
}
```

If you need the "latest" value, you can pass a function: `setValue((prev) => prev + 1)`. It's rarely needed in this course, but it's good to recognize.

Kinds of state and how to change them

State	Typical update

boolean toggle	setVisible(!visible)
number	setCount(count + 1), setIndex((index + 1) % COLORS.length)
string	setName(event.target.value)
Union (QuestionType)	setType(type === "a" ? "b" : "a")
Cycle through values	setHoliday(NEXT[holiday]) with a Record
Array: add	setItems([...items, newItem])
Array: remove	setItems(items.filter(i => i !== target))
Array: change one	setQs(qs.map(q => q.id === id ? {...q, x} : q))
Array: clear	setItems([])
Object: change a field	setMovie({ ...movie, rating: 1 })
Record: bump a key	setVotes({ ...votes, [key]: votes[key] + 1 })

NEVER MUTATE STATE

```
team.push(newMember); setTeam(team); // ✗ same array reference, so React may not re-render
team = []; // ✗ can't reassign const, and React wouldn't know
anyway
setTeam([...team, newMember]); // ✓ new array
setTeam([]); // ✓ new empty array
```

Derived values: compute them, don't store them

If something can be calculated from state, calculate it during render. Don't make a second piece of state for it.

```
const [answer, setAnswer] = useState<string>("");
const isCorrect = answer === expectedAnswer; // ✓ derived, always in sync
// const [isCorrect, setIsCorrect] = useState(false); // ✗ extra state that can get out of sync
```

Disabling buttons with state

```
<Button onClick={startQuiz} disabled={inProgress || attempts === 0}>Start Quiz</Button>
<Button onClick={stopQuiz} disabled={!inProgress}>Stop Quiz</Button>
```

Each component instance has its own state

If `App` renders `<Counter />` three times, you get three separate counters. State belongs to the instance, not the file. To **share** state between components, you lift it up ([Section 7](#)).

Your state tasks, solved and explained

These are your own solutions from the repo. All 52 tests pass.

Counter

src/components/Counter.tsx

src/components/Counter.tsx

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

export function Counter(): React.JSX.Element {
  const [value, setValue] = useState<number>(0);
  return (
    <span>
      <Button
        onClick={() => {
          setValue(1 + value);
        }}
      >
        Add One
      </Button>{" "}
      to {value}.
    </span>
  );
}
```

`{" "}` is an explicit space. JSX drops whitespace at line breaks.

RevealAnswer

src/components/RevealAnswer.tsx

A button toggles whether the answer "42" is visible. Hidden at first.

src/components/RevealAnswer.tsx

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

export function RevealAnswer(): React.JSX.Element {
  const [visible, setVisible] = useState<boolean>(false);

  return (
    <div>
      <Button
        onClick={() => {
          setVisible(!visible);
        }}
        >
        Reveal Answer
      </Button>
      {visible && <div>42</div>}
    </div>
  );
}
```

ChangeType

src/components/ChangeType.tsx

State is a `QuestionType` starting at short answer; the button flips it; show "Multiple Choice" or "Short Answer".

src/components/ChangeType.tsx

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";
import { QuestionType } from "../interfaces/question";

export function ChangeType(): React.JSX.Element {
  const [type, setType] = useState<QuestionType>("short_answer_question");

  function changeType(): void {
    setType(
      type === "short_answer_question" ?
        "multiple_choice_question"
      : "short_answer_question",
    );
  }

  return (
    <div>
      <Button onClick={changeType}>Change Type</Button>
      <div>
        {type === "multiple_choice_question" ?
          "Multiple Choice"
        : "Short Answer"}
      </div>
    </div>
  );
}
```

Union-type state works just like any other state. The `<QuestionType>` type makes sure you can only set one of the two allowed strings.

StartAttempt `src/components/StartAttempt.tsx`

4 attempts. Start: uses an attempt and begins the quiz (disabled while in progress or at 0). Stop: ends it (disabled when not in progress). Mulligan: +1 attempt (disabled in progress).

src/components/StartAttempt.tsx

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

export function StartAttempt(): React.JSX.Element {
  const [attempts, setAttempts] = useState<number>(4);
  const [inProgress, setInProgress] = useState<boolean>(false);

  function startQuiz(): void {
    setInProgress(true);
    setAttempts(attempts - 1);
  }

  return (
    <div>
      <span>Attempts: {attempts}</span>
    </div>
  );
}
```

```

    <div>
      <Button
        onClick={startQuiz}
        disabled={inProgress || attempts === 0}
      >
        Start Quiz
      </Button>
      <Button
        onClick={() => {
          setInProgress(false);
        }}
        disabled={!inProgress}
      >
        Stop Quiz
      </Button>
      <Button
        onClick={() => {
          setAttempts(attempts + 1);
        }}
        disabled={inProgress}
      >
        Mulligan
      </Button>
    </div>
  </div>
);
}

```

Two pieces of state that depend on each other. The work is in the `disabled` expressions. Read each one out loud as "disabled when..."

TwoDice `src/components/TwoDice.tsx`

Two dice start different (3 and 4). Each button rerolls one die. Same value → "You Win!", except both 1s → "You Lose!"

`src/components/TwoDice.tsx`

```

import React, { useState } from "react";
import { Button } from "react-bootstrap";

export function d6(): number {
  return 1 + Math.floor(Math.random() * 6);
}

export function TwoDice(): React.JSX.Element {
  const [leftDie, setLeftDie] = useState<number>(3);
  const [rightDie, setRightDie] = useState<number>(4);

  return (
    <div>
      <span data-testid="left-die">{leftDie}</span>
      <span> and </span>
      <span data-testid="right-die">{rightDie}</span>
    </div>
  );
}

```

```

        <Button
          onClick={() => {
            setLeftDie(d6());
          }}
        >
          Roll Left
        </Button>
        <Button
          onClick={() => {
            setRightDie(d6());
          }}
        >
          Roll Right
        </Button>
      </div>
      {leftDie === 1 && rightDie === 1 && <div>You Lose!</div>}
      {leftDie === rightDie && leftDie !== 1 && <div>You Win!</div>}
    </div>
  );
}

```

CycleHoliday

src/components/CycleHoliday.tsx

Show a holiday emoji; one button advances alphabetically, one by date in the year.

src/components/CycleHoliday.tsx

```

import React, { useState } from "react";
import { Button } from "react-bootstrap";

type Holiday = "🌲" | "🐰" | "🎃" | "🍷" | "💖";

const NEXT_BY_ALPHABET: Record<Holiday, Holiday> = {
  "🌲": "🐰",
  "🐰": "🎃",
  "🎃": "🍷",
  "🍷": "💖",
  "💖": "🌲",
};

const NEXT_BY_YEAR: Record<Holiday, Holiday> = {
  "💖": "🐰",
  "🐰": "🎃",
  "🎃": "🍷",
  "🍷": "🌲",
  "🌲": "💖",
};

export function CycleHoliday(): React.JSX.Element {
  const [holiday, setHoliday] = useState<Holiday>("🎃");

  return (
    <div>
      <div>Holiday: {holiday}</div>
      <Button
        onClick={() => {

```

```

        setHoliday(NEXT_BY_ALPHABET[holiday]);
      }}
    >
      Advance by Alphabet
    </Button>
    <Button
      onClick={() => {
        setHoliday(NEXT_BY_YEAR[holiday]);
      }}
    >
      Advance by Year
    </Button>
  </div>
);
}

```

A `Record<Holiday, Holiday>` maps "current" → "next". It's cleaner than a long chain of ternaries.

Components, props and lifting state

Props ("properties") are the inputs a parent passes to a child component, written like HTML attributes. A component receives them as **one object** parameter. Props are **read-only**: a child never changes its props.

Three ways to type props (all correct)

```
// A. Interface + props object
interface IDCardProps {
  firstName: string;
  lastName: string;
}
function IDCard(props: IDCardProps): React.JSX.Element {
  return <p>{props.firstName} {props.lastName}</p>;
}

// B. Interface + destructuring (common)
function IDCard({ firstName, lastName }: IDCardProps): React.JSX.Element {
  return <p>{firstName} {lastName}</p>;
}

// C. Inline type + destructuring (YOUR REPO'S STYLE, e.g. CheckAnswer)
function IDCard({
  firstName,
  lastName,
}: {
  firstName: string;
  lastName: string;
}): React.JSX.Element {
  return <p>{firstName} {lastName}</p>;
}

// Using it (the parent passes the props):
<IDCard firstName="Ada" lastName="Bart" />
```

Note the pattern `(( { a, b } : { a: string; b: number } ))`. The first `{ }` deconstructs and the second is the type. Inside a type, fields are separated by `;`.

Passing functions as props

Props can be functions. That's how a child tells its parent that something happened. The prop type is a **function type**:

```
setVisible: (newVisibility: boolean) => void // a setter for boolean state
setPosition: (newPosition: number) => void // a setter for number state
setTeam: (newTeam: string[]) => void // a setter for array state
onDelete: (id: number) => void // a custom callback
showHide: () => void // no arguments
```

A setter from `useState<number>` fits the type `(newValue: number) => void`, so you can pass it straight down.

Lifting state up

State lives in **one** component. If two components need the same state (one shows it, the other changes it), move the `useState` **up to their nearest common parent**. Then pass the value down to the one that displays it, and the setter (or a function that calls it) down to the one that changes it.

Parent owns `useState` → value (props) → Display child

Parent passes the setter → setter (props) → Button child calls it →

Parent state changes, so **both** children re-render

State flows down through props. Actions flow up through callback functions.

lifting state: the full pattern

```
function RevealButton({
  visible,
  setVisible,
}): {
  visible: boolean;
  setVisible: (newVisibility: boolean) => void;
}: React.JSX.Element {
  return (
    <Button
      onClick={() => {
        setVisible(!visible);
      }}
    >
      {visible ? "Hide" : "Show"}
    </Button>
  );
}

function Answer({ visible }: { visible: boolean }): React.JSX.Element {
  return <div>Answer: {visible ? "42" : "SECRET"}</div>;
}

export function App(): React.JSX.Element {
  const [visible, setVisible] = useState<boolean>(false); // state lives in the PARENT
  return (
    <div>
      <RevealButton visible={visible} setVisible={setVisible} />
      <Answer visible={visible} />
    </div>
  );
}
```

Instead of passing the raw setter, you can pass a ready-made function: `<RevealButton showHide={() => { setVisible(!visible); }} />` with the prop typed `showHide: () => void`. Both approaches are accepted.

The four anti-patterns (your "bad components")

Anti-pattern	Why it's wrong	Fix	Task
Mutating state: <code>team.push(x)</code> , <code>team = []</code>	Same reference, so React doesn't see a change. Reassigning a <code>const</code> isn't allowed.	<code>setTeam([...team, x])</code> , <code>setTeam([])</code>	ChooseTeam
State trapped in one child while a sibling needs it	Siblings can't see each other's state	Lift <code>useState</code> to the parent and pass props	ColoredBox
<code>useState</code> at the top of a file, shared by <code>import</code>	Breaks the rules of hooks (outside a component)	Put it in the parent component and pass it as props	DoubleHalf
Calling a component like a function: <code>const box = MoveableBox()</code> , then <code>box.position</code>	A component returns JSX, not an object with fields, so you can't reach its state. Hooks would also run in the wrong component.	Lift the state and render <code><MoveableBox position={position} /></code>	ShoveBox

One more from the textbook: **don't define a component inside another component**. It gets recreated on every render and loses its state. Define every component at the top level of the file.

Fixing the bad components, step by step

ChooseTeam: stop mutating, pass arguments

`src/bad-components/ChooseTeam.tsx`

Clicking a person adds them to the team (no duplicates). "Clear Team" empties it. The tests expect 7 buttons and list items in click order.

What's broken in the starter:

starter (broken) · `src/bad-components/ChooseTeam.tsx`

```
export function ChooseTeam(): React.JSX.Element {
  const [allOptions, setAllOptions] = useState<string[]>(PEOPLE);
  const [team, setTeam] = useState<string[]>([]);

  function chooseMember() {
    /*
     * if (!team.includes(newMember)) {
       team.push(newMember);
     */
  }
}
```

```

    }
    */
  }

  function clearTeam() {
    /*
     team = [];
    */
  }

  return (
    <div>
      <h3>Choose Team</h3>
      <Row>
        <Col>
          {allOptions.map((option: string) => (
            <div key={option} style={{ marginBottom: "4px" }}>
              Add{" "}
              <Button onClick={chooseMember} size="sm">
                {option}
              </Button>
            </div>
          ))}
        </Col>
        <Col>
          <strong>Team:</strong>
          {team.map((member: string) => (
            <li key={member}>{member}</li>
          ))}
          <Button onClick={clearTeam}>Clear Team</Button>
        </Col>
      </Row>
    </div>
  );
}

```

1. `chooseMember` has no way to know *which* person was clicked, because `onClick={chooseMember}` passes nothing useful.
2. `team.push(...)` mutates state.
3. `team = []` reassigns a `const` and bypasses the setter.

✓ fixed · all 6 tests pass

```

import React, { useState } from "react";
import { Button, Row, Col } from "react-bootstrap";

const PEOPLE = [
  "Alan Turing",
  "Grace Hopper",
  "Ada Lovelace",
  "Charles Babbage",
  "Barbara Liskov",
  "Margaret Hamilton",
];

export function ChooseTeam(): React.JSX.Element {

```

```

const [allOptions] = useState<string[]>(PEOPLE);
const [team, setTeam] = useState<string[]>([]);

function chooseMember(newMember: string) {
  if (!team.includes(newMember)) {
    setTeam([...team, newMember]);
  }
}

function clearTeam() {
  setTeam([]);
}

return (
  <div>
    <h3>Choose Team</h3>
    <Row>
      <Col>
        {allOptions.map((option: string) => (
          <div key={option} style={{ marginBottom: "4px" }}>
            Add{ " " }
            <Button
              onClick={() => {
                chooseMember(option);
              }}
              size="sm"
            >
              {option}
            </Button>
          </div>
        ))}
      </Col>
      <Col>
        <strong>Team:</strong>
        {team.map((member: string) => (
          <li key={member}>{member}</li>
        ))}
        <Button onClick={clearTeam}>Clear Team</Button>
      </Col>
    </Row>
  </div>
);
}

```

The key move is `onClick={() => { chooseMember(option); }}`. The arrow function **closes over** the current `option` from the `map`, so each button remembers its own person. (`setAllOptions` was dropped because it was unused and the linter complains. Keeping `const [allOptions, setAllOptions] = ...` would work too.)

ColoredBox: lift state to the parent

`src/bad-components/ColoredBox.tsx`

A "Next Color" button cycles `red` → `blue` → `green` → `red`; the preview box and the text show the current color.

What's broken: `colorIndex` lives inside the button component (`ChangeColor`), but `ColorPreview` and the text in `ColoredBox` need it too. They're stuck showing `DEFAULT_COLOR_INDEX` forever.

✓ fixed · all 3 tests pass

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

export const COLORS = ["red", "blue", "green"];
const DEFAULT_COLOR_INDEX = 0;

function ChangeColor({
  colorIndex,
  setColorIndex,
}): {
  colorIndex: number;
  setColorIndex: (newIndex: number) => void;
}: React.JSX.Element {
  return (
    <Button
      onClick={() => {
        setColorIndex((1 + colorIndex) % COLORS.length);
      }}
    >
      Next Color
    </Button>
  );
}

function ColorPreview({ colorIndex }: { colorIndex: number }): React.JSX.Element {
  return (
    <div
      data-testid="colored-box"
      style={{
        width: "50px",
        height: "50px",
        backgroundColor: COLORS[colorIndex],
        display: "inline-block",
        verticalAlign: "bottom",
        marginLeft: "5px",
      }}
    ></div>
  );
}

export function ColoredBox(): React.JSX.Element {
  const [colorIndex, setColorIndex] = useState<number>(DEFAULT_COLOR_INDEX);
  return (
    <div>
      <h3>Colored Box</h3>
      <span>The current color is: {COLORS[colorIndex]}</span>
      <div>
        <ChangeColor
          colorIndex={colorIndex}
          setColorIndex={setColorIndex}
        ></ChangeColor>
        <ColorPreview colorIndex={colorIndex}></ColorPreview>
      </div>
    </div>
  );
}
```

```

    </div>
  );
}

```

Steps: (1) move `useState` into `ColoredBox`, the common parent. (2) Give `ChangeColor` props for the value and setter. (3) Give `ColorPreview` a prop for the value. (4) Replace every `DEFAULT_COLOR_INDEX` used for display with `colorIndex . % COLORS.length` wraps $2 \rightarrow 0$.

DoubleHalf: no hooks outside components

`src/bad-components/DoubleHalf.tsx`

Value starts at 10. "Double" and "Halve" buttons change it.

What's broken: the state is created at the top level of a separate file and imported:

starter (broken) · `src/bad-components/DoubleHalfState.tsx`

```

import { useState } from "react";

export const [dhValue, setDhValue] = useState<number>(10);

```

This breaks rule 1 of hooks, so it crashes. (That's why `App.tsx` has `<DoubleHalf>` commented out.) State can't be shared through `import`.

✓ fixed · all 5 tests pass (the `DoubleHalfState` import is deleted)

```

import React, { useState } from "react";
import { Button } from "react-bootstrap";

function Doubler({
  dhValue,
  setDhValue,
}: {
  dhValue: number;
  setDhValue: (newValue: number) => void;
}): React.JSX.Element {
  return (
    <Button
      onClick={() => {
        setDhValue(2 * dhValue);
      }}
      >
      Double
    </Button>
  );
}

function Halver({
  dhValue,
  setDhValue,
}: {
  dhValue: number;

```

```

    setDhValue: (newValue: number) => void;
  }): React.JSX.Element {
    return (
      <Button
        onClick={() => {
          setDhValue(0.5 * dhValue);
        }}
      >
        Halve
      </Button>
    );
  }

export function DoubleHalf(): React.JSX.Element {
  const [dhValue, setDhValue] = useState<number>(10);
  return (
    <div>
      <h3>Double Half</h3>
      <div>
        The current value is: <span>{dhValue}</span>
      </div>
      <Doubler dhValue={dhValue} setDhValue={setDhValue}></Doubler>
      <Halver dhValue={dhValue} setDhValue={setDhValue}></Halver>
    </div>
  );
}

```

ShoveBox: don't call components as functions

src/bad-components/ShoveBox.tsx

Box starts at `marginLeft 10px`; each "Shove the Box" click moves it 4px right.

What's broken:

starter (broken)

```

export function ShoveBox(): React.JSX.Element {
  const box = MoveableBox();

  return (
    <div>
      <h3>Shove Box</h3>
      /* <span>The box is at: {box.position}</span>
      <div>
        <ShoveBoxButton
          position={box.position}
          setPosition={box.setPosition}
        ></ShoveBoxButton>
        {box}
      </div> */
    </div>
  );
}

```

`MoveableBox()` returns JSX, so there's no `box.position` to read. Components aren't objects with fields. The state inside `MoveableBox` is unreachable from outside.

✓ fixed · all 3 tests pass

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

function ShoveBoxButton({
  position,
  setPosition,
}): {
  position: number;
  setPosition: (newPosition: number) => void;
}): React.JSX.Element {
  return (
    <Button
      onClick={() => {
        setPosition(4 + position);
      }}
      >
      Shove the Box
    </Button>
  );
}

function MoveableBox({ position }: { position: number }): React.JSX.Element {
  return (
    <div
      data-testid="moveable-box"
      style={{
        width: "50px",
        height: "50px",
        backgroundColor: "lightblue",
        border: "1px solid blue",
        display: "inline-block",
        verticalAlign: "bottom",
        marginLeft: position + "px",
      }}
    ></div>
  );
}

export function ShoveBox(): React.JSX.Element {
  const [position, setPosition] = useState<number>(10);

  return (
    <div>
      <h3>Shove Box</h3>
      <span>The box is at: {position}</span>
    </div>
    <ShoveBoxButton
      position={position}
      setPosition={setPosition}
    ></ShoveBoxButton>
    <MoveableBox position={position}></MoveableBox>
  </div>
</div>
```

```
    );  
  }
```

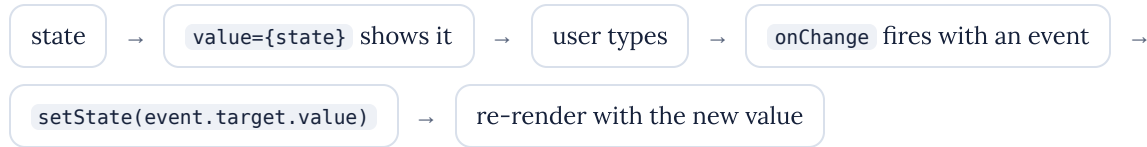
The fix is the same move as ColoredBox: the parent (ShoveBox) owns `position` . It passes `position` and `setPosition` to the button and `position` to the box, and renders the box with JSX tags.

HOW TO FIX ANY BROKEN COMPONENT ON THE EXAM

1. **Find the state.** Which component calls `useState` ?
2. **Find who needs it.** List every component that reads or changes it.
3. **Move it up** to the lowest component that contains all of them.
4. **Pass it down:** the value to readers, the setter (typed `(x: T) => void`) to changers.
5. **Check updates:** no `push` , `=` , `splice` or `sort` on state. Use spread, `filter` or `map` , then call the setter.
6. **Check rendering:** components as `<Tags />` , handlers passed not called, a `key` on every mapped element.

Forms: controlled components

A **form element** (textbox, checkbox, dropdown...) lets the user edit state. In React Hooks you write **controlled components**: the element *shows* the state, and every change the user makes goes *back into* the state through `onChange`.



The anatomy of a form field (React Bootstrap)

```

import React, { useState } from "react";
import { Form } from "react-bootstrap";

export function MovieName(): React.JSX.Element {
  const [name, setName] = useState<string>("Alien");

  function updateName(event: React.ChangeEvent<HTMLInputElement>) {
    setName(event.target.value);
  }

  return (
    <div>
      <Form.Group controlId="formMovieName">           {/* groups a label + control */}
        <Form.Label>Name:</Form.Label>                 {/* the visible label */}
        <Form.Control value={name} onChange={updateName} />  {/* the textbox */}
      </Form.Group>
      <div>The movie is "{name}".</div>
    </div>
  );
}

```

- `Form.Group` with `controlId` links the label to the input, which helps accessibility and lets tests find it by label.
- `Form.Control` is the input. Without a `type` it's a single-line text box.
- `value={name}` plus `onChange={updateName}` make it controlled. If you forget `onChange`, the box becomes read-only.

The event types: memorize this table

Element	JSX	Event type	Read	State type
Textbox	<code><Form.Control /></code>	<code>React.ChangeEvent<HTMLInputElement></code>	<code>.target.value</code>	<code>string</code>
Number box	<code><Form.Control type="number" /></code>	<code>React.ChangeEvent<HTMLInputElement></code>	<code>.target.value</code>	<code>string</code> (parse it later!)

Textarea	<code><Form.Control as="textarea" rows={3} /></code>	<code>React.ChangeEvent<HTMLTextA reaElement></code>	<code>.target.val ue</code>	string
Checkbox	<code><Form.Check type="checkbox" /></code>	<code>React.ChangeEvent<HTMLInput Element></code>	<code>.target.che cked</code>	boolean
Switch	<code><Form.Check type="switch" /></code>	<code>React.ChangeEvent<HTMLInput Element></code>	<code>.target.che cked</code>	boolean
Radio group	<code><Form.Check type="radio" /> (several)</code>	<code>React.ChangeEvent<HTMLInput Element></code>	<code>.target.val ue</code>	string
Select	<code><Form.Select><option/>...</code>	<code>React.ChangeEvent<HTMLSelec tElement></code>	<code>.target.val ue</code>	string

VALUE VS CHECKED

Text, number, textarea, select and radio use **value**. Checkbox and switch use **checked**, both in the JSX (`checked={isHappy}`) and in the handler (`event.target.checked`). For a radio button, `checked` is *calculated*: `checked={emotion === "happy"}`.

The textbook also shows a handy alias that works for both inputs and textareas: `type ChangeEvent = React.ChangeEvent<HTMLTextAreaElement | HTMLInputElement>;`

Textbox

Shown above. Handlers can also be written inline:

```
<Form.Control
  value={name}
  onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
    setName(event.target.value);
  }}
/>
```

Number box: the value is still a string

```
const [released, setReleased] = useState<string>("2022"); // STRING state
const year = parseInt(released); // parse when you need a number
const previousYear = parseInt(released) - 1 || 0; // NaN falls back to 0

<Form.Group controlId="formMovieReleased">
  <Form.Label>Released:</Form.Label>
  <Form.Control
    type="number"
    value={released}
    onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
      setReleased(event.target.value);
    }}
  />
</Form.Group>
```

Why a string? The box can briefly hold "" (empty) or a half-typed value. Keep what the user typed, and parse it where you use it. Check for blanks with `isNaN(parseInt(x))`.

Textarea

```
<Form.Control
  as="textarea"
  rows={3}
  value={description}
  onChange={(event: React.ChangeEvent<HTMLTextAreaElement>) => {
    setDescription(event.target.value);
  }}
/>
```

Checkbox and switch

```
const [isHappy, setIsHappy] = useState<boolean>(true);

<Form.Check
  type="checkbox" // or type="switch": same logic, looks like a toggle
  id="is-happy-check" // unique id so the label is clickable
  label="Happy?"
  checked={isHappy}
  onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
    setIsHappy(event.target.checked);
  }}
/>
<div>The user is {isHappy ? "happy" : "not happy"}.</div>
```

`Form.Check` brings its own label, so you don't need a `Form.Group` or `Form.Label`. A switch renders inside a `div` with the class `form-switch`, and your EditMode test checks for it.

Radio buttons

Every radio button in a group shares the same `name`. Each has its own `value`. Each is `checked` when the state equals its value.

```
const [emotion, setEmotion] = useState<string>("happy");

function updateEmotion(event: React.ChangeEvent<HTMLInputElement>) {
  setEmotion(event.target.value);
}

<Form.Check type="radio" name="emotions" id="emotion-happy" label="Happy"
  value="happy" checked={emotion === "happy"} onChange={updateEmotion} />
<Form.Check type="radio" name="emotions" id="emotion-sad" label="Sad"
  value="sad" checked={emotion === "sad"} onChange={updateEmotion} />

// Same thing generated from an array (do this when there are many):
const EMOTIONS = ["happy", "sad", "angry"];
{EMOTIONS.map((e: string) => (
  <Form.Check
    inline                // side by side
    key={e}
    type="radio"
    name="emotions"
    id={"emotion-" + e}
    label={e}
    value={e}
    checked={emotion === e}
    onChange={updateEmotion}
  />
))}
```

Multiple checkboxes (array state)

```
const [emotions, setEmotions] = useState<string[]>(["happy"]);

function updateEmotion(event: React.ChangeEvent<HTMLInputElement>) {
  const emotion = event.target.value;
  if (emotions.includes(emotion)) {
    setEmotions(emotions.filter((e: string): boolean => e !== emotion)); // uncheck → remove
  } else {
    setEmotions([...emotions, emotion]); // check → add
  }
}

<Form.Check type="checkbox" id="emo-happy" label="Happy" value="happy"
  checked={emotions.includes("happy")} onChange={updateEmotion} />
```

Select (dropdown)

```
const COLORS = ["red", "blue", "green"];
const [favorite, setFavorite] = useState<string>(COLORS[0]);

function updateFavorite(event: React.ChangeEvent<HTMLSelectElement>) {
  setFavorite(event.target.value);
}

<Form.Group controlId="favoriteColors">
  <Form.Label>What is your favorite color?</Form.Label>
  <Form.Select value={favorite} onChange={updateFavorite}>
    {COLORS.map((color: string) => (
      <option key={color} value={color}>
        {color}
      </option>
    ))}
  </Form.Select>
</Form.Group>
```

The `value` goes on `Form.Select` (what's selected). Each `<option>` has its own `value` and a `key`. `event.target.value` is always a **string**, so if your options are ids, use `parseInt(event.target.value)`.

Your form tasks, solved and explained

Your repo still has the empty starter code for these. Every solution below passes all of its tests (41/41 for components and forms together). Write each on paper first.

CheckAnswer `src/form-components/CheckAnswer.tsx`

A textbox. Show ✓ if what's typed equals the `expectedAnswer` prop, otherwise ✗ (including at the start).

✓ solution · 6/6 tests

```
import React, { useState } from "react";
import { Form } from "react-bootstrap";

export function CheckAnswer({
  expectedAnswer,
}): {
  expectedAnswer: string;
}): React.JSX.Element {
  const [answer, setAnswer] = useState<string>("");

  function updateAnswer(event: React.ChangeEvent<HTMLInputElement>) {
    setAnswer(event.target.value);
  }

  return (
    <div>
      <h3>Check Answer</h3>
      <Form.Group controlId="formCheckAnswer">
        <Form.Label>Your answer:</Form.Label>
        <Form.Control value={answer} onChange={updateAnswer} />
      </Form.Group>
      <div>{answer === expectedAnswer ? "✓" : "✗"}</div>
    </div>
  );
}
```

The ✓/✗ is derived from state plus the prop. No extra state is needed. The test finds the input by role "textbox", which is what `Form.Control` renders.

GiveAttempts

src/form-components/GiveAttempts.tsx

Start with 3 attempts. A number box plus two buttons: "use" (-1, disabled at 0) and "gain" (+ the typed amount; a blank amount does nothing).

✓ solution · 5/5 tests

```
import React, { useState } from "react";
import { Button, Form } from "react-bootstrap";

export function GiveAttempts(): React.JSX.Element {
  const [attemptsLeft, setAttemptsLeft] = useState<number>(3);
  const [requested, setRequested] = useState<string>("");

  function gainAttempts() {
    const amount = parseInt(requested);
    if (!isNaN(amount)) {
      setAttemptsLeft(attemptsLeft + amount);
    }
  }

  return (
    <div>
```

```

    <h3>Give Attempts</h3>
    <Form.Group controlId="formRequestAttempts">
      <Form.Label>Attempts to add:</Form.Label>
      <Form.Control
        type="number"
        value={requested}
        onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
          setRequested(event.target.value);
        }}
      />
    </Form.Group>
    <Button
      onClick={() => {
        setAttemptsLeft(attemptsLeft - 1);
      }}
      disabled={attemptsLeft <= 0}
    >
      use
    </Button>
    <Button onClick={gainAttempts}>gain</Button>
    <div>Attempts left: {attemptsLeft}</div>
  </div>
);
}

```

- Two pieces of state: the attempts **number**, and the typed amount as a **string**.
- `parseInt("")` is `NaN`, so `if (!isNaN(amount))` handles the blank case. `parseInt(requested) || 0` works too.
- The test finds the buttons by name with `/use/i` and `/gain/i`, so the button text has to contain those words.

EditMode src/form-components/EditMode.tsx

A switch toggles edit mode (off at first). Not editing: show "NAME is a student" / "NAME is not a student" (default name "Your Name", student true). Editing: show a textbox for the name and a "student" checkbox instead.

✓ solution · 5/5 tests

```

import React, { useState } from "react";
import { Form } from "react-bootstrap";

export function EditMode(): React.JSX.Element {
  const [editMode, setEditMode] = useState<boolean>(false);
  const [name, setName] = useState<string>("Your Name");
  const [isStudent, setIsStudent] = useState<boolean>(true);

  return (
    <div>
      <h3>Edit Mode</h3>
      <Form.Check
        type="switch"
        id="is-edit-mode"

```

```

        label="Edit Mode"
        checked={editMode}
        onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
            setEditMode(event.target.checked);
        }}
    />
    {editMode ?
        <div>
            <Form.Group controlId="formEditName">
                <Form.Label>Name:</Form.Label>
                <Form.Control
                    value={name}
                    onChange={(
                        event: React.ChangeEvent<HTMLInputElement>,
                    ) => {
                        setName(event.target.value);
                    }}
                />
            </Form.Group>
            <Form.Check
                type="checkbox"
                id="is-student-check"
                label="Student"
                checked={isStudent}
                onChange={(
                    event: React.ChangeEvent<HTMLInputElement>,
                ) => {
                    setIsStudent(event.target.checked);
                }}
            />
        </div>
        :
        <div>
            {name} is {isStudent ? "a student" : "not a student"}
        </div>
    }
</div>
);
}

```

Three pieces of state: `editMode`, `name` and `isStudent`. A ternary picks which view to show. Both the switch and the checkbox read `event.target.checked`. The test finds the checkbox by name `/student/i`, so its label must say "Student".

MultipleChoiceQuestion

`src/form-components/MultipleChoiceQuestion.tsx`

A dropdown of the options prop; ✓ if the choice equals `expectedAnswer`, otherwise ✗. The first option starts selected.

✓ solution · 6/6 tests

```
import React, { useState } from "react";
import { Form } from "react-bootstrap";

export function MultipleChoiceQuestion({
  options,
  expectedAnswer,
}): {
  options: string[];
  expectedAnswer: string;
}): React.JSX.Element {
  const [choice, setChoice] = useState<string>(options[0]);

  function updateChoice(event: React.ChangeEvent<HTMLSelectElement>) {
    setChoice(event.target.value);
  }

  return (
    <div>
      <h3>Multiple Choice Question</h3>
      <Form.Group controlId="formMultipleChoice">
        <Form.Label>Choose an answer:</Form.Label>
        <Form.Select value={choice} onChange={updateChoice}>
          {options.map((option: string) => (
            <option key={option} value={option}>
              {option}
            </option>
          ))}
        </Form.Select>
      </Form.Group>
      <div>{choice === expectedAnswer ? "✓" : "✗"}</div>
    </div>
  );
}
```

The initial state comes from a prop: `useState<string>(options[0])` . The select has role `"combobox"` in the tests.

ChangeColor

src/form-components/ChangeColor.tsx

At least 8 radio buttons of colors. A box with data-testid "colored-box" shows the chosen color's name as text and as its background color.

✓ solution · 2/2 tests

```
import React, { useState } from "react";
import { Form } from "react-bootstrap";

const COLORS = [
  "red",
  "blue",
  "green",
  "orange",
```

```

    "purple",
    "cyan",
    "magenta",
    "white",
  ];

  export function ChangeColor(): React.JSX.Element {
    const [color, setColor] = useState<string>(COLORS[0]);

    function updateColor(event: React.ChangeEvent<HTMLInputElement>) {
      setColor(event.target.value);
    }

    return (
      <div>
        <h3>Change Color</h3>
        {COLORS.map((c: string) => (
          <Form.Check
            inline
            key={c}
            type="radio"
            name="colors"
            id={"color-check-" + c}
            label={c}
            value={c}
            checked={color === c}
            onChange={updateColor}
          />
        ))}
        <div>
          You have chosen{" "}
          <span
            data-testid="colored-box"
            style={{ backgroundColor: color }}
          >
            {color}
          </span>
        </div>
      </div>
    );
  }

```

The radios come from a `COLORS.map`, all sharing `name="colors"`. The box uses `style={{ backgroundColor: color }}`. The test compares the box's text and background to each radio's `value`, so `value`, `text` and `color` all come from the same string.

Reading specs and tests

Your task tests say exactly which elements a component must contain. Exam questions describe the same things in words ("a number entry box and two buttons"). Knowing which JSX produces which **role** turns a spec into code quickly.

Roles: what each element counts as

Test asks for role...	You write
"button"	<code><Button></code> (the text inside is its <i>name</i>)
"textbox"	<code><Form.Control /></code> or <code>as="textarea"</code>
"spinbutton"	<code><Form.Control type="number" /></code>
"checkbox"	<code><Form.Check type="checkbox" /></code> or <code>type="switch"</code>
"radio"	<code><Form.Check type="radio" /></code>
"combobox"	<code><Form.Select></code>
"listitem"	<code></code>
"heading"	<code><h1> ... <h6></code>
<code>getById("x")</code>	any element with <code>data-testid="x"</code>
<code>getByLabelText(/Name/)</code>	a control linked to a label (<code>controlId</code> or <code>Form.Check label</code>)

Query words

- `getBy...` : exactly one must exist, otherwise the test fails.
- `queryBy...` : returns `null` if missing. Used to check that something is *not* shown.
- `getAllBy...` / `queryAllBy...` : return arrays, for counting (`toHaveLength(7)`).
- `{ name: /use/i }` : matches the accessible name (button text or label), case-insensitive.

Matchers you'll see

`toBeInTheDocument()`, `toHaveLength(n)`, `toEqual(x)` (same contents), `toBe(x)` (same value or reference), `toHaveStyle({ backgroundColor: "red" })`, `toHaveTextContent("red")`, `toHaveValue("42")`, `toBeChecked()`, `toBeDisabled()`, `toHaveClass("form-switch")`.

How a React test reads (Given, When, Then)

```
from src/form-components/GiveAttempts.test.tsx
```

```
test("(2 pts) You can use attempts", async () => {  
  const use = screen.getByRole("button", { name: /use/i }); // GIVEN a "use" button  
  await act(async () => {  
    use.click(); // WHEN it's clicked  
  });  
  expect(screen.getByText(/2/i)).toBeInTheDocument(); // THEN "2" is showing  
});
```

Reading it backwards tells you what to build: a button whose text contains "use", and the attempt count shown as text. `userEvent.type(box, "10")` types into a box. `userEvent.selectOptions(select, "2")` picks an option. Both fire your `onChange`.

TURN A WORD SPEC INTO ELEMENTS

"There is a number entry box and two buttons" → one `Form.Control type="number"` + two `Buttons`. "There is one checkbox and no textboxes" → a `Form.Check` and nothing typed until edit mode. "At least 8 radio buttons" → an 8-item array mapped to `Form.Check type="radio"`.

Writing a component on paper: the recipe

Use the same seven steps every time. Doing them in order keeps you from forgetting imports, types or the `return`.

1. **Imports.** `import React, { useState } from "react";` plus `import { Button, Form } from "react-bootstrap";` and any interfaces.
2. **Signature.** `export function Name({ props }: { types }): React.JSX.Element {`
3. **State.** For each thing that changes, write `const [x, setX] = useState<T>(initial);`. Form text and numbers are strings, checkboxes are booleans.
4. **Derived values.** Anything you can compute from state or props goes in a plain `const`.
5. **Handlers.** Small functions that call setters with new values (spread, `map`, `filter`).
6. **Return JSX.** One wrapper `<div>`, the controls, lists with `key`, and conditionals with `&&` or `? :`.
7. **Check.** Keys ✓, handlers passed not called ✓, `value` or `checked` plus `onChange` ✓, event types ✓, nothing mutated ✓, closing tags ✓.

Worked example: everything at once

"Write a `QuizEditor` component that receives `initialQuestions: Question[]`. It lists every question's name with a button that toggles that question's published status (button text "Publish" or "Unpublish"). A checkbox "Show only published" filters the list. Below the list, show the total points of published questions."

Planning it out:

- State: the question list (`Question[]` , starting from the prop) and the checkbox (`boolean`).
- Handler: `togglePublished(id)` , which is pattern 8 inside `setQuestions` .
- Derived: `visible` (filtered or not) and `publishedPoints` (filter + reduce).
- JSX: a checkbox, then `visible.map` into `<li key={q.id}>` with a button, then the total.

QuizEditor.tsx • verified with tests

```
import React, { useState } from "react";
import { Button, Form } from "react-bootstrap";
import { Question } from "../interfaces/question";

export function QuizEditor({
  initialQuestions,
}: {
  initialQuestions: Question[];
}): React.JSX.Element {
  const [questions, setQuestions] = useState<Question[]>(initialQuestions);
  const [onlyPublished, setOnlyPublished] = useState<boolean>(false);

  function togglePublished(targetId: number): void {
    setQuestions(
      questions.map(
        (question: Question): Question =>
          question.id === targetId ?
```

```

        { ...question, published: !question.published }
        :   question,
      ),
    );
  }

  const visible =
    onlyPublished ?
      questions.filter((question: Question): boolean => question.published)
    :   questions;

  const publishedPoints = questions
    .filter((question: Question): boolean => question.published)
    .reduce((sum: number, question: Question): number => sum + question.points, 0);

  return (
    <div>
      <Form.Check
        type="checkbox"
        id="only-published-check"
        label="Show only published"
        checked={onlyPublished}
        onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
          setOnlyPublished(event.target.checked);
        }}
      />
      <ul>
        {visible.map((question: Question) => (
          <li key={question.id}>
            {question.name}{ " "}
            <Button
              size="sm"
              onClick={() => {
                togglePublished(question.id);
              }}
            >
              {question.published ? "Unpublish" : "Publish"}
            </Button>
          </li>
        ))}
      </ul>
      <div>Published points: {publishedPoints}</div>
    </div>
  );
}

```

Notice how much of this is Section 4 (nested data) running inside React. That's why the data parts come first.

Practice: data functions

These are new problems in the same style as your tasks, all using `Question` and `Answer`. Write each on paper with full types, then check your work in the Answer Key at the back. Every solution was checked with tests, including a check that the input array is never mutated.

P1 • `getQuestionsWorth`

Given `questions: Question[]` and `points: number`, return only the questions worth exactly that many points.

[Answer key → P1](#)

P2 • `countMultipleChoice`

Return how many questions are multiple choice.

[Answer key → P2](#)

P3 • `addBonusPoints`

Return a new array where every question is worth `bonus` more points.

[Answer key → P3](#)

P4 • `unpublishById`

Return a new array where the question with `targetId` is unpublished; all others are unchanged.

[Answer key → P4](#)

P5 • `removeOption`

For the question with `targetId`, remove the option at `optionIndex`. Other questions are unchanged.

[Answer key → P5](#)

P6 • `averagePoints`

Return the average points per question, or 0 for an empty list.

[Answer key → P6](#)

P7 • `getLongestName`

Return the longest question name (the first one wins ties), or `""` if there are no questions.

[Answer key → P7](#)

P8 • makeExpectedLookup

Return a `Record<number, string>` from each question's id to its expected answer.

Answer key → P8

P9 • moveToEnd

Move the question with `targetId` to the end of the list. If it's not found, return a copy unchanged.

Answer key → P9

P10 • gradeAnswers

Given `questions` and `answers: Answer[]`, return new answers marked `submitted: true`, with `correct` set by comparing `text` to the matching question's `expected` (ignoring case and whitespace). An answer with no matching question is incorrect.

Answer key → P10

P11 • addOptionToAllMC

Add `newOption` to the options of every multiple-choice question. Each changed question must get its own new options array.

Answer key → P11

P12 • insertAfter

Insert `newQuestion` right after the question with `targetId`. If it's not found, add it to the end.

Answer key → P12

Practice: components

Each component below was rendered and clicked through in tests. Write the whole file on paper, imports included.

R1 • LightSwitch state basics

A "Flip" button and the text "The light is on." / "The light is off." It starts off.

[Answer key → R1](#)

R2 • ShoppingList array state + textbox

A textbox and an "Add" button. Add puts the trimmed text in the list (skip blanks and duplicates) and clears the box. Each list item has a "Remove ITEM" button. Show "You have N items."

[Answer key → R2](#)

R3 • TemperatureConverter number box

A number box for Fahrenheit. Show "That is C degrees Celsius" (rounded), or "Enter a temperature" when the box is blank.

[Answer key → R3](#)

R4 • Scoreboard lifting state

Three components: `ScoreDisplay` shows "Score: N". `ScoreButtons` has "Point" (+1) and "Reset" (0). `Scoreboard` is the parent that connects them.

[Answer key → R4](#)

R5 • QuizEditor nested state + checkbox

The worked example from [Section 10](#). Try it from the spec alone before looking back.

R6 • Poll radio + Record state

Radio buttons for "vanilla", "chocolate" and "strawberry" (vanilla selected at first) and a "Vote" button that adds one vote to the selected flavor. List each flavor as "flavor: N".

[Answer key → R6](#)

R7 • RenameQuestion select + textbox + update by id

Receives `initialQuestions`. A dropdown picks a question by id. A textbox edits the *selected* question's name live. Below that, list all question names.

Answer key → R7

Spot the bug, then trace it

Graders love these. Find what's wrong, say *why* in one sentence, and write the fix.

B1

```
export function Toggle(): React.JSX.Element {
  const [on, setOn] = useState<boolean>(false);
  return <Button onClick={setOn(!on)}>{on ? "On" : "Off"}</Button>;
}
```

Answer key → B1

B2

```
function addItem(): void {
  items.push(newItem);
  setItems(items);
}
```

Answer key → B2

B3

```
<ul>
  {questions.map((q: Question) => <li>{q.name}</li>)}
</ul>
```

Answer key → B3

B4

```
export function Extra({ showExtra }: { showExtra: boolean }): React.JSX.Element {
  if (showExtra) {
    const [bonus, setBonus] = useState<number>(0);
  }
  // ...
}
```

Answer key → B4

B5

```
const [done, setDone] = useState<boolean>(false);
<Form.Check type="checkbox" label="Done" value={done}
  onChange={e: React.ChangeEvent<HTMLInputElement>} => { setDone(e.target.value); }} />
```

Answer key → B5

B6

```
function publish(id: number): void {
  setQuestions(questions.map((q: Question): Question => {
    if (q.id === id) {
      q.published = true;
    }
    return q;
  }));
}
```

Answer key → B6

B7

```
const reset = questions.map((q: Question): Question => { ...q, points: 0 });
```

Answer key → B7

B8

```
const [amount, setAmount] = useState<number>(0);
<Form.Control type="number" value={amount}
  onChange={e: React.ChangeEvent<HTMLInputElement>} => { setAmount(e.target.value); }} />
```

Answer key → B8

B9

```
function updateChoice(event: React.ChangeEvent<HTMLInputElement>) {
  setChoice(event.target.value);
}
<Form.Select value={choice} onChange={updateChoice}>...</Form.Select>
```

Answer key → B9

B10

```
export function Parent(): React.JSX.Element {  
  function Child(): React.JSX.Element {  
    const [n, setN] = useState<number>(0);  
    return <Button onClick={() => { setN(n + 1); }}>{n}</Button>;  
  }  
  return <Child />;  
}
```

Answer key → B10

Trace it: what's shown?

T1

```
const [n, setN] = useState<number>(5);  
<Button onClick={() => { setN(n * 2); }}>A</Button>  
<Button onClick={() => { setN(n - 3); }}>B</Button>  
<span>{n}</span>  
// Click A, then B, then A. What does the span show?
```

Answer key → T1

T2

```
function bump(): void {  
  setCount(count + 1);  
  setCount(count + 1);  
}  
// count starts at 0. bump runs 3 times (3 separate clicks). Final count?
```

Answer key → T2

T3

```
[3, 8, 1, 10]  
  .filter((n: number): boolean => n > 2)  
  .map((n: number): number => n * 10)  
  .reduce((acc: number, n: number): number => acc + n, 0);
```

Answer key → T3

T4

```
const words = ["a", "bb", "ccc"];
words.map((w: string, i: number): string => w + i).join("-");
words.find((w: string): boolean => w.length > 1);
words.findIndex((w: string): boolean => w.length > 5);
words.some((w: string): boolean => w === "bb");
words.every((w: string): boolean => w.length < 3);
```

Answer key → T4

T5

```
const q1 = { id: 1, options: ["a"] };
const q2 = { ...q1 };
q2.options.push("b");
const q3 = { ...q1, options: [...q1.options, "c"] };
// What are q1.options, q2.options, q3.options?
```

Answer key → T5

Mistakes checklist

Run through this list before you move on from any exam question. These are the errors that cost the most points on handwritten React code.

DATA

- Used `push`, `splice`, `sort` or `x.field = ...` on the input or on state
- Forgot `( )` around an object returned from an arrow function
- Put the spread *after* the changed field (`{ name, ...q }`)
- Copied the outer object but not the nested array
- No starting value in `reduce`
- `find` can return `undefined`, but you returned it where `null` was expected
- Compared with `==` or checked `=== NaN`
- Missing parameter or return types
- Forgot the `return` in a braced arrow function

REACT

- Missing `import React, { useState } from "react"` or the Bootstrap import
- `useState` outside the component, or inside an `if` or loop
- `onClick={f()}` instead of `onClick={f}` or `{() => { f(x); }}`
- No `key` on mapped elements
- Checkbox or switch using `value` instead of `checked`
- Wrong event type (`HTMLSelectElement` for select, `HTMLTextAreaElement` for textarea)
- Number-box state typed as `number`
- Radio group without a shared `name`, or with `checked` not compared to state
- Returning two sibling elements without a wrapper
- Calling a component as a function, or defining one inside another
- Lowercase component name, or `class` instead of `className`

Cheat sheet

The five templates, plus the syntax you'll reach for most. On Day 5, write this whole page from memory.

Template 1 • Update one item by id

```
setQuestions(
  questions.map((q: Question): Question =>
    q.id === targetId ? { ...q, name: newName } : q,
  ),
);
```

Template 2 • State and a button

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

export function Thing(): React.JSX.Element {
  const [count, setCount] = useState<number>(0);
  return (
    <div>
      <Button
        onClick={() => {
          setCount(count + 1);
        }}
        disabled={count >= 10}
      >
        Add
      </Button>
      <span>{count}</span>
      {count === 10 && <div>Max!</div>}
    </div>
  );
}
```

Template 3 • Controlled inputs

```
// text / number: string state, read .value
<Form.Group controlId="formName">
  <Form.Label>Name:</Form.Label>
  <Form.Control
    value={name}
    onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
      setName(event.target.value);
    }}
  />
</Form.Group>

// checkbox / switch: boolean state, read .checked
<Form.Check type="switch" id="edit-switch" label="Edit" checked={edit}
  onChange={(event: React.ChangeEvent<HTMLInputElement>) => { setEdit(event.target.checked); }}
/>

// radio: string state, checked is calculated
{OPTIONS.map((opt: string) => (
  <Form.Check key={opt} type="radio" name="group" id={"opt-" + opt} label={opt}
    value={opt} checked={choice === opt}
    onChange={(event: React.ChangeEvent<HTMLInputElement>) => { setChoice(event.target.value); }}
  />
))}

// select: HTMLSelectElement
<Form.Select value={choice}
  onChange={(event: React.ChangeEvent<HTMLSelectElement>) => { setChoice(event.target.value); }}
>>
  {OPTIONS.map((opt: string) => (
    <option key={opt} value={opt}>{opt}</option>
  ))}
</Form.Select>
```

Template 4 • Lifting state

```
function Child({ value, setValue }: { value: number; setValue: (newValue: number) => void }):
  React.JSX.Element {
  return <Button onClick={() => { setValue(value + 1); }}>{value}</Button>;
}

export function Parent(): React.JSX.Element {
  const [value, setValue] = useState<number>(0);
  return <Child value={value} setValue={setValue} />;
}
```

Template 5 • Rendering a list

```
<ul>
  {questions.map((q: Question) => (
    <li key={q.id}>
      {q.name}{" "}
      <Button onClick={() => { remove(q.id); }}>Remove</Button>
    </li>
  ))}
</ul>
```

Array methods at a glance

You want...	Write
transform all	<code>arr.map(x => ...)</code>
keep some / remove some	<code>arr.filter(x => keep?)</code>
one number or object from many	<code>arr.reduce((acc, x) => ..., start)</code>
first match (or undefined)	<code>arr.find(x => ...)</code>
position (or -1)	<code>arr.findIndex(x => ...)</code>
all / any?	<code>arr.every(...)</code> / <code>arr.some(...)</code>
contains a value?	<code>arr.includes(v)</code>
add to end / start	<code>[...arr, x]</code> / <code>[x, ...arr]</code>
insert at i	<code>const c = [...arr]; c.splice(i, 0, x);</code>
replace at i	<code>arr.map((x, j) => j === i ? newX : x)</code>
remove at i	<code>arr.filter((x, j) => j !== i)</code>
count matches	<code>arr.filter(...).length</code>
sum a field	<code>arr.reduce((s, x) => s + x.points, 0)</code>
to a string	<code>arr.join(", ")</code>
sorted copy	<code>[...arr].sort((a, b) => a - b)</code>

Objects at a glance

You want...	Write
copy + change	<code>{ ...obj, field: value }</code>
toggle a boolean field	<code>{ ...obj, published: !obj.published }</code>

add to a nested array	<code>{ ...obj, options: [...obj.options, x] }</code>
clear a nested array	<code>{ ...obj, options: [] }</code>
dynamic key	<code>{ ...rec, [key]: rec[key] + 1 }</code>
pull out fields	<code>const { name, points } = q;</code>
lookup table type	<code>Record<string, number></code>

Glossary

Every term you might see in a question. To quiz yourself, cover the definitions and explain each term out loud.

TypeScript

JavaScript plus static types. It's checked before running and compiled into JavaScript.

Type annotation

The `: type` after a variable, parameter or function, e.g. `(n: number): string`.

Union type

A value that can be one of several types or literals: `string | null`, `"a" | "b"`.

Type alias

A name for a type: `type QuestionType = "..." | "...";`

Interface

A named description of an object's fields and their types.

Optional field

A field marked `?` in an interface, meaning it may be missing.

Record

`Record<K, V>`: an object used as a key → value lookup table.

Arrow function

A compact function: `(x: number): number => x * 2`.

Callback

A function passed to another function to be called later, like the function you give `map` or `onClick`.

Higher-order function

A function that takes or returns a function: `map`, `filter`, `reduce` ...

Predicate

A callback that returns a boolean (used by `filter`, `find`, `every`, `some`).

Accumulator

The running result carried through `reduce`.

Reference

What an object or array variable actually holds: a pointer to the data. `===` compares references for objects and arrays.

Mutation

Changing an existing array or object in place (`push`, `obj.x = 1`). Avoid it in this course.

Immutability

Never changing data. You make a new copy with the change instead.

Spread (...)

Unpacks an array's items or an object's fields into a new array or object.

Shallow copy

A copy of the top level only. Nested arrays and objects are still shared.

Deep copy

A copy where every nested level is copied too.

Destructuring

Pulling fields or items into variables: `const { a } = obj;` , `const [x, setX] = ...`

Template literal

A backtick string with `${expression}` slots.

Ternary

`cond ? a : b` , an if/else expression that produces a value.

Truthy / falsy

Falsy values are `false` , `0` , `""` , `null` , `undefined` and `NaN` . Everything else is truthy.

NaN

"Not a Number". It comes from a failed parse or bad math. Check it with `isNaN()` .

JSON

A text format for data. `JSON.stringify` converts an object to text and `JSON.parse` converts it back.

React

A library for building UIs from components that re-render when their state changes.

Component

A capitalized function that returns JSX. It's used like a tag: `<Counter />` .

JSX / TSX

HTML-like syntax inside JavaScript or TypeScript files.

Element

One piece of JSX (`<div>...</div>`). A component returns an element.

Fragment

`<>...</>` : groups elements without adding a wrapper to the page.

Props

Read-only inputs a parent passes to a child component.

State

Data a component remembers between renders. Changing it triggers a re-render.

Hook

A special React function starting with `use` (like `useState`). It can only be called at the top of a component.

useState

The hook that creates state: `const [value, setValue] = useState<T>(initial);`

Setter

The function from `useState` that schedules a new state value and a re-render.

Render / re-render

React calling your component function to get the JSX, and then updating the page.

Rules of hooks

Call hooks only inside components, and never in conditions, loops or nested functions.

Event handler

A function run when something happens (`onClick`, `onChange`).

Event object

What `onChange` passes to your handler. `event.target` is the element, with `.value` or `.checked`.

Controlled component

A form element whose value comes from state and whose changes go back into state through `onChange`.

Lifting state up

Moving state to the nearest common parent so several children can share it through props.

One-way data flow

Data goes down through props, and changes go up through callbacks.

Derived value

A value computed from state or props during render instead of stored in its own state.

Key

A unique prop on each element produced by `map`, so React can track list items.

Conditional rendering

Showing JSX only sometimes, with `&&` or a ternary.

Closure

A function that remembers variables from where it was created, like `() => chooseMember(option)` inside a `map`.

MVC

Model (state), View (JSX), Controller (handlers that call setters).

React Bootstrap

A library of pre-styled React components: `Button`, `Form`, `Row`, `Col`, `Container`.

controlId

The prop on `Form.Group` that links a label to its control.

DOM

The page as a tree of HTML elements. React updates it for you.

Linting / Prettier

Tools that check code for problems and style (ESLint), and format it automatically (Prettier).

Node / npm

Node runs JavaScript outside the browser. npm installs packages.

Unit test

Code that checks one function or component (`test(...)`, `expect(...)`).

data-testid

An attribute that lets tests find an element with `getByTestId`.

Answer key

Solutions to every practice problem. Try each problem on paper before you look here.

Data functions

P1 • getQuestionsWorth

```
export function getQuestionsWorth(questions: Question[], points: number): Question[] {
  return questions.filter((question: Question): boolean => question.points === points);
}
```

P2 • countMultipleChoice

```
export function countMultipleChoice(questions: Question[]): number {
  return questions.filter(
    (question: Question): boolean => question.type === "multiple_choice_question",
  ).length;
}
```

P3 • addBonusPoints

```
export function addBonusPoints(questions: Question[], bonus: number): Question[] {
  return questions.map(
    (question: Question): Question => ({ ...question, points: question.points + bonus }),
  );
}
```

P4 • unpublishById

```
export function unpublishById(questions: Question[], targetId: number): Question[] {
  return questions.map(
    (question: Question): Question =>
      question.id === targetId ? { ...question, published: false } : question,
  );
}
```

P5 • removeOption

```
export function removeOption(
  questions: Question[],
  targetId: number,
  optionIndex: number,
): Question[] {
  return questions.map(
    (question: Question): Question =>
      question.id === targetId ?
```

```

        {
          ...question,
          options: question.options.filter(
            (option: string, index: number): boolean => index !== optionIndex,
          ),
        }
      : question,
    );
  }
}

```

The inner `filter` uses its second parameter, the index, to drop exactly one position.

P6 • averagePoints

```

export function averagePoints(questions: Question[]): number {
  if (questions.length === 0) {
    return 0;
  }
  const total = questions.reduce(
    (sum: number, question: Question): number => sum + question.points,
    0,
  );
  return total / questions.length;
}

```

P7 • getLongestName

```

export function getLongestName(questions: Question[]): string {
  return questions.reduce(
    (longest: string, question: Question): string =>
      question.name.length > longest.length ? question.name : longest,
    "",
  );
}

```

P8 • makeExpectedLookup

```

export function makeExpectedLookup(questions: Question[]): Record<number, string> {
  return questions.reduce(
    (lookup: Record<number, string>, question: Question): Record<number, string> => ({
      ...lookup,
      [question.id]: question.expected,
    }),
    {},
  );
}

```

Alternative: `Object.fromEntries(questions.map(q => [q.id, q.expected]))`.

P9 • moveToEnd

```
export function moveToEnd(questions: Question[], targetId: number): Question[] {
  const target = questions.find((question: Question): boolean => question.id === targetId);
  if (target === undefined) {
    return [...questions];
  }
  return [
    ...questions.filter((question: Question): boolean => question.id !== targetId),
    target,
  ];
}
```

P10 • gradeAnswers

```
export function gradeAnswers(questions: Question[], answers: Answer[]): Answer[] {
  return answers.map((answer: Answer): Answer => {
    const question = questions.find(
      (q: Question): boolean => q.id === answer.questionId,
    );
    const correct =
      question !== undefined &&
      answer.text.trim().toLowerCase() === question.expected.trim().toLowerCase();
    return { ...answer, submitted: true, correct: correct };
  });
}
```

P11 • addOptionToAllMC

```
export function addOptionToAllMC(questions: Question[], newOption: string): Question[] {
  return questions.map(
    (question: Question): Question =>
      question.type === "multiple_choice_question" ?
        { ...question, options: [...question.options, newOption] }
        : question,
  );
}
```

P12 • insertAfter

```
export function insertAfter(
  questions: Question[],
  targetId: number,
  newQuestion: Question,
): Question[] {
  const index = questions.findIndex((q: Question): boolean => q.id === targetId);
  if (index === -1) {
    return [...questions, newQuestion];
  }
  const copy = [...questions];
  copy.splice(index + 1, 0, newQuestion);
}
```

```
    return copy;
  }
```

Components

R1 · LightSwitch

LightSwitch.tsx

```
import React, { useState } from "react";
import { Button } from "react-bootstrap";

export function LightSwitch(): React.JSX.Element {
  const [isOn, setIsOn] = useState<boolean>(false);

  return (
    <div>
      <Button
        onClick={() => {
          setIsOn(!isOn);
        }}
      >
        Flip
      </Button>
      <div>The light is {isOn ? "on" : "off"}.</div>
    </div>
  );
}
```

R2 · ShoppingList

ShoppingList.tsx

```
import React, { useState } from "react";
import { Button, Form } from "react-bootstrap";

export function ShoppingList(): React.JSX.Element {
  const [items, setItems] = useState<string[]>([]);
  const [newItem, setNewItem] = useState<string>("");

  function addItem(): void {
    const cleaned = newItem.trim();
    if (cleaned !== "" && !items.includes(cleaned)) {
      setItems([...items, cleaned]);
    }
    setNewItem("");
  }

  function removeItem(target: string): void {
    setItems(items.filter((item: string): boolean => item !== target));
  }

  return (
    <div>
      <Form.Group controlId="formNewItem">
```

```

        <Form.Label>New item:</Form.Label>
        <Form.Control
          value={newItem}
          onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
            setNewItem(event.target.value);
          }}
        />
      </Form.Group>
      <Button onClick={addItem}>Add</Button>
      <ul>
        {items.map((item: string) => (
          <li key={item}>
            {item}{ " "}
            <Button
              size="sm"
              onClick={() => {
                removeItem(item);
              }}
            >
              Remove {item}
            </Button>
          </li>
        ))}
      </ul>
      <div>You have {items.length} items.</div>
    </div>
  );
}

```

R3 • TemperatureConverter

TemperatureConverter.tsx

```

import React, { useState } from "react";
import { Form } from "react-bootstrap";

export function TemperatureConverter(): React.JSX.Element {
  const [fahrenheit, setFahrenheit] = useState<string>("");

  const parsed = parseInt(fahrenheit);
  const celsius = Math.round(((parsed - 32) * 5) / 9);

  return (
    <div>
      <Form.Group controlId="formFahrenheit">
        <Form.Label>Fahrenheit:</Form.Label>
        <Form.Control
          type="number"
          value={fahrenheit}
          onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
            setFahrenheit(event.target.value);
          }}
        />
      </Form.Group>
      <div>
        {isNaN(parsed) ?
          "Enter a temperature"
          : `That is ${celsius} degrees Celsius`}
      </div>
    </div>
  );
}

```

```

        </div>
      </div>
    );
  }

```

R4 • Scoreboard

Scoreboard.tsx

```

import React, { useState } from "react";
import { Button } from "react-bootstrap";

function ScoreDisplay({ score }: { score: number }): React.JSX.Element {
  return <div>Score: {score}</div>;
}

function ScoreButtons({
  score,
  setScore,
}: {
  score: number;
  setScore: (newScore: number) => void;
}): React.JSX.Element {
  return (
    <div>
      <Button
        onClick={() => {
          setScore(score + 1);
        }}
      >
        Point
      </Button>
      <Button
        onClick={() => {
          setScore(0);
        }}
      >
        Reset
      </Button>
    </div>
  );
}

export function Scoreboard(): React.JSX.Element {
  const [score, setScore] = useState<number>(0);
  return (
    <div>
      <ScoreDisplay score={score}></ScoreDisplay>
      <ScoreButtons score={score} setScore={setScore}></ScoreButtons>
    </div>
  );
}

```

R6 • Poll

Poll.tsx

```

import React, { useState } from "react";
import { Button, Form } from "react-bootstrap";

const FLAVORS = ["vanilla", "chocolate", "strawberry"];

export function Poll(): React.JSX.Element {
  const [choice, setChoice] = useState<string>(FLAVORS[0]);
  const [votes, setVotes] = useState<Record<string, number>>({
    vanilla: 0,
    chocolate: 0,
    strawberry: 0,
  }));

  function vote(): void {
    setVotes({ ...votes, [choice]: votes[choice] + 1 });
  }

  return (
    <div>
      {FLAVORS.map((flavor: string) => (
        <Form.Check
          key={flavor}
          type="radio"
          name="flavors"
          id={"flavor-" + flavor}
          label={flavor}
          value={flavor}
          checked={choice === flavor}
          onChange={(event: React.ChangeEvent<HTMLInputElement>) => {
            setChoice(event.target.value);
          }}
        />
      ))}
      <Button onClick={vote}>Vote</Button>
      <ul>
        {FLAVORS.map((flavor: string) => (
          <li key={flavor}>
            {flavor}: {votes[flavor]}
          </li>
        ))}
      </ul>
    </div>
  );
}

```

R7 • RenameQuestion

RenameQuestion.tsx

```

import React, { useState } from "react";
import { Form } from "react-bootstrap";
import { Question } from "../interfaces/question";

export function RenameQuestion({
  initialQuestions,
}: {
  initialQuestions: Question[];

```

```

}): React.JSX.Element {
  const [questions, setQuestions] = useState<Question[]>(initialQuestions);
  const [selectedId, setSelectedId] = useState<number>(initialQuestions[0].id);

  const selected = questions.find(
    (question: Question): boolean => question.id === selectedId,
  );

  function rename(event: React.ChangeEvent<HTMLInputElement>): void {
    setQuestions(
      questions.map(
        (question: Question): Question =>
          question.id === selectedId ?
            { ...question, name: event.target.value }
            : question,
      ),
    );
  }

  return (
    <div>
      <Form.Group controlId="formPickQuestion">
        <Form.Label>Pick a question:</Form.Label>
        <Form.Select
          value={selectedId}
          onChange={(event: React.ChangeEvent<HTMLSelectElement>) => {
            setSelectedId(parseInt(event.target.value));
          }}
        >
          {questions.map((question: Question) => (
            <option key={question.id} value={question.id}>
              {question.name}
            </option>
          ))}
        </Form.Select>
      </Form.Group>
      <Form.Group controlId="formRename">
        <Form.Label>New name:</Form.Label>
        <Form.Control value={selected !== undefined ? selected.name : ""} onChange=
{rename} />
      </Form.Group>
      <ul>
        {questions.map((question: Question) => (
          <li key={question.id}>{question.name}</li>
        ))}
      </ul>
    </div>
  );
}

```

The select's value is a number id, but `event.target.value` is always a string, hence the `parseInt`. The textbox shows a **derived** value (the selected question's name), and typing runs pattern 8.

Spot the bug and trace it

B1

`setOn(!on)` is **called during render** instead of on click, which causes an infinite re-render loop. Fix: `onClick={() => { setOn(!on); }}`.

B2

It mutates state and passes the **same reference** back, so React may not re-render. Fix:

```
setItems([...items, newItem]);
```

B3

Missing `key`. Fix: `<li key={q.id}>{q.name}</li>`.

B4

A hook inside an `if` breaks the rules of hooks, because the call order would change between renders. Always call `useState` at the top of the component, and use the condition when you *render*.

B5

Checkboxes use `checked`, not `value`. Also, `e.target.value` is a string, so it's a type error for boolean state. Fix: `checked={done}` and `setDone(e.target.checked)`.

B6

The array is new, but the object inside it was **mutated**, which changes the old state too. Fix: `q.id === id ? { ...q, published: true } : q`.

B7

The object literal needs **parentheses**, otherwise `{` starts a function body (a syntax error). Fix: `=> ({ ...q, points: 0 })`.

B8

`e.target.value` is a **string**, so it can't go into number state. Fix: use `useState<string>("")` and `parseInt` where you need the number. (`setAmount(parseInt(e.target.value))` type-checks, but a blank box becomes `NaN`.)

B9

Wrong event type: a select fires `React.ChangeEvent<HTMLSelectElement>`. TypeScript rejects this handler.

B10

A component is defined **inside** another component, so it gets recreated on every render and its state resets. Move `Child` to the top level of the file.

T1

5 → A → **10** → B → 7 → A → **14**.

T2

3. Both setters in one click read the same old `count`, so each click adds only 1.

T3

filter → `[3, 8, 10]` → map → `[30, 80, 100]` → reduce → **210**.

T4

`"a0-bb1-ccc2"`, `"bb"`, `-1`, `true`, `false`.

T5

q1: `["a", "b"]`. q2: `["a", "b"]` (the **same array** as q1, since spread is shallow). q3: `["a", "b", "c"]` (its own new array, copied after the push).